

Synergy Vision

Machine Learning Básico



Índice general

Índice de cuadros	VII
Índice de figuras	IX
Prefacio	XIII
1. Introducción	1
1.1. Clasificación de los Aprendizajes	2
1.2. Organización del texto	4
1.3. Conjunto de Bases de Datos utilizadas	5
2. Ambientes de trabajo para el científico de datos	7
2.1. Primer encuentro con los lenguajes	8
2.2. Anaconda	12
2.3. Librerías	13
2.3.1. Instalando librerías en <i>R</i>	13
2.3.2. Instalando librerías en <i>Python</i>	15
3. Uso y preparación de los datos	17
3.1. Lectura de datos	17
3.1.1. Lectura de datos con <i>R</i>	17
3.1.2. Lectura de datos con <i>Python</i>	20
3.2. Tratamiento previo de los datos	22
3.2.1. Missing Values o valores faltantes	23
3.3. Identificación y tratamientos de Outliers o puntos atípicos	29
3.3.1. Identificación de puntos atípicos	29
3.3.2. Detección de outliers en <i>R</i>	32
3.3.3. Detección de outliers en <i>Python</i>	35
4. Regresión Lineal	41

4.1.	Identificando relación entre variables	41
4.1.1.	Identificando relación entre variables con <i>R</i>	43
4.1.2.	Identificando relación entre variables con <i>Python</i>	45
4.2.	Transformando variables	47
4.2.1.	Aplicando transformaciones de variables en <i>R</i>	48
4.2.2.	Aplicando transformaciones de variables en <i>Python</i>	48
4.3.	Eliminando el efecto de colinealidad entre variables independientes.	50
4.4.	Eliminación de las variables categóricas	51
4.4.1.	Eliminación de variables en <i>R</i>	51
4.4.2.	Eliminación de variables en <i>Python</i>	51
4.5.	Aplicando el modelo de regresión lineal	52
4.5.1.	Aplicando el modelo de regresión lineal en <i>R</i>	52
4.5.2.	Aplicando el modelo de regresión lineal en <i>Python</i> :	54
4.6.	Cómo interpretar los resultados del modelo	56
4.6.1.	Interpretar con <i>summary</i> en <i>R</i> :	56
4.6.2.	Interpretar con <i>summary</i> en <i>Python</i>	58
4.7.	Realizando predicciones con nuestro modelo	59
4.7.1.	Realizando predicciones con <i>R</i>	59
4.7.2.	Realizando predicciones con <i>Python</i>	59
5.	Regresión logística	61
5.1.	Supuestos de la regresión logística	61
5.2.	Multicolinealidades entre variables independientes	62
5.3.	Detectar correlación entre variables categóricas	62
5.3.1.	Prueba chi-cuadrado en <i>R</i>	63
5.3.2.	Prueba chi-cuadrado en <i>Python</i>	64
5.4.	Detectar correlación entre variables categóricas y variables cuantitativas	64
5.4.1.	Medida de <i>d</i> de Cohen en <i>R</i>	65
5.4.2.	Medida de <i>d</i> de Cohen en <i>Python</i>	65
5.5.	Codificando variables categóricas	66
5.5.1.	Creación de variables auxiliares en <i>R</i>	66

5.5.2. Creación de variables auxiliares en <i>Python</i>	67
5.6. Aplicando el modelo de regresión logística	68
5.6.1. Modelo logístico en <i>R</i>	68
5.6.2. Modelo logístico en <i>Python</i>	71
6. Análisis de clúster	75
6.1. Supuestos del análisis de clúster.	75
6.2. Medidas o similitudes	76
6.3. Métodos jerárquicos y no jerárquicos	76
6.3.1. Métodos jerárquicos	77
6.3.2. Métodos no jerárquicos	77
6.4. Elección del número de grupos	78
6.4.1. Dendrograma	78
6.4.2. Gráfico de codo	78
6.5. Análisis de clúster en <i>R</i>	79
6.5.1. Método no jerárquico	79
6.5.2. Método jerárquico	82
6.6. Análisis de clúster en <i>Python</i>	84
6.6.1. Método no jerárquico	84
6.6.2. Método jerárquico	86
7. Análisis de Componentes Principales	89
7.1. Correlación o Covarianza:	89
7.2. Supuestos estadísticos	90
7.3. <i>PCA</i> en <i>R</i>	90
7.3.1. Librerías y datos	90
7.3.2. Cálculo de componentes	91
7.3.3. Funciones gráficas	92
7.4. <i>PCA</i> en <i>Python</i>	92
7.4.1. Librerías y datos	92
7.4.2. Cálculo de componentes	94
7.4.3. Funciones Gráficas	95
8. Árboles de decisión y bosques aleatorios	97
8.1. Árboles de decisión	97
8.1.1. Árboles de decisión en <i>R</i>	97
8.1.2. Árboles de decisión en <i>Python</i>	99
8.2. Bosques aleatorios	101

8.3. Bosques aleatorios en <i>R</i>	101
8.4. Bosques aleatorios en <i>Python</i>	103
Apéndice	107
A. Herramientas de software	107
A.1. <i>R</i> y paquetes de <i>R</i>	107
A.2. Pandoc	108

Índice de cuadros



Índice de figuras

2.1.	7
2.2.	9
2.3.	10
2.4.	11
2.5.	11
2.6.	12
2.7.	15
2.8.	16
2.9.	16
3.1.	18
3.2.	19
3.3.	20
3.4.	21
3.5.	22
3.6.	23
3.7.	25
3.8.	25
3.9.	25
3.10.	26
3.11.	26
3.12.	27
3.13.	27
3.14.	27
3.15.	28
3.16.	28
3.17.	28
3.18.	30
3.19.	31
3.20.	32

3.21.	33
3.22.	33
3.23.	34
3.24.	34
3.25.	35
3.26.	36
3.27.	36
3.28.	37
3.29.	37
3.30.	38
3.31.	38
3.32.	39
3.33.	39
3.34.	40
3.35.	40
4.1.	42
4.2.	43
4.3.	43
4.4.	44
4.5.	44
4.6.	45
4.7.	45
4.8.	45
4.9.	46
4.10.	46
4.11.	47
4.12.	48
4.13.	49
4.14.	49
4.15.	50
4.16.	51
4.17.	52
4.18.	53
4.19.	54
4.20.	54
4.21.	55

4.22.	55
4.23.	56
4.24.	57
4.25.	58
4.26.	59
4.27.	60
5.1.	62
5.2.	63
5.3.	63
5.4.	63
5.5.	64
5.6.	65
5.7.	65
5.8.	66
5.9.	67
5.10.	67
5.11.	68
5.12.	69
5.13.	69
5.14.	69
5.15.	70
5.16.	71
5.17.	72
5.18.	73
5.19.	73
6.1.	79
6.2.	80
6.3.	80
6.4.	81
6.5.	81
6.6.	82
6.7.	83
6.8.	83
6.9.	84
6.10.	85
6.11.	85

6.12.	85
6.13.	86
6.14.	86
6.15.	87
6.16.	87
7.1.	91
7.2.	91
7.3.	91
7.4.	92
7.5.	92
7.6.	93
7.7.	93
7.8.	94
7.9.	94
7.10.	95
7.11.	95
7.12.	96
8.1.	98
8.2.	98
8.3.	99
8.4.	100
8.5.	100
8.6.	100
8.7.	101
8.8.	102
8.9.	102
8.10.	103
8.11.	104
8.12.	104
8.13.	105

Prefacio



La versión en línea de este libro se comparte bajo la licencia Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License¹.

¿Por qué leer este libro?

Este libro aspira a que las empresas iberoamericanas tengan un acceso fácil y en su propio idioma a una tecnología que les permitirá el manejo estratégico de sus datos para soportar mejor la toma de decisiones sobre sus negocios. Para ello, el libro aportará con ejemplos muy diversos del mundo empresarial, las herramientas básicas del Machine Learning, haciendo énfasis en las aplicaciones prácticas con ayuda de los softwares R y Python, sin requerir mayores complicaciones matemáticas o computacionales. Los lenguajes de programación Python y R, son sin duda, los más utilizados a la hora de construir modelos de inteligencia artificial.

El material expuesto en el libro se concentra en precisar las técnicas más comunes usadas en Machine Learning, para que las empresas o individuos interesados, tengan un punto de partida sólido sobre el cual ir construyendo un conjunto de herramientas que puedan utilizar en cualquier momento. El lector podrá aprender las principales librerías y dominar la sintaxis para la construcción de modelos.

¹<http://creativecommons.org/licenses/by-nc-sa/4.0/>

Contar con este conocimiento es una gran ventaja, pues el gran avance de internet en la última década ha hecho crecer de forma exponencial la cantidad de datos que se producen por la interacción con sus usuarios. Así, poder crear modelos que permitan sacar ventaja a partir de estos datos, se ha vuelto una meta imprescindible para la mayoría de las empresas en el mundo actual, dado que en dichos datos se encuentran patrones que siguen clientes y usuarios. Los individuos o empresas que logren develar estos patrones, tendrán una enorme ventaja estratégica sobre sus competidores en el mundo actual. Esperamos que este libro pueda ayudarlos en esa tarea.

Estructura del libro

Este libro se divide en 7 capítulos, en los cuales se especificarán las instrucciones técnicas para poder acceder a los programas y funciones a utilizar. Cada sección estará compuesta por una sección teórica, seguida de la sintaxis requerida para la construcción de los modelos. En la mayoría de los casos esta información es proporcionada a través de imágenes donde el lector observará cómo aplicar la sintaxis de forma adecuada.

La secuencia de ejemplos de uso de las herramientas siempre se presentará primero en R y luego en Phyton.

El propósito de este material es ofrecer una experiencia de aprendizaje distinta y enfocada en el estudiante de forma que realmente aprenda y practique con mucha intensidad. La idea es cambiar el modelo de clases magistrales y ofrecer una experiencia más centrada en el estudiante y menos centrado en el profesor. Para los temas más técnicos y avanzados es necesario trabajar de la mano con el estudiante y asistirlo en el proceso de aprendizaje con prácticas guiadas, material en línea e interactivo, videos, evaluación continua de brechas y entendimiento, entre otros, para procurar el dominio de la materia.

Información sobre los programas y convenciones

A lo largo de este material estaremos usando las palabras *paquetes y librerías*, las cuales se refieren a lo mismo, la única diferencia es la preferencia de uso entre las comunidades que utilizan los diversos lenguajes de programación. De igual forma, usaremos la palabra función o método, las cuales si bien presentan algunas diferencias entre ellas dependiendo del lenguaje utilizado, tienen en general para este curso, el mismo significado. Por tal razón las usaremos indistintamente, dado que las variaciones solo son relevantes mencionarlas en un curso avanzado de programación.

Las definiciones en el libro irán en **negrita**, mientras que los nombres de paquetes y funciones se escribirán en *cursivas*.

Acerca del Autor

Synergy Vision es una empresa que se especializa en proveer la Formación, la Tecnología y la Consultoría en las áreas de Especulación (Trading), Inversión, Economía, Finanzas y Riesgo.

Ofrecemos Formación de Alto Nivel para prácticas de Consultoría y Ejercicio Profesional Especializado para alcanzar Certificaciones reconocidas a nivel mundial como el CFA, CMT y FRM.

Aplicamos técnicas que conjugan lo que hoy en día se conoce como Data Science que consiste en la aplicación de herramientas de las Matemáticas, Estadísticas y las Ciencias de la Computación, además de las técnicas de Minería de Datos (Datamining) y Aprendizaje de máquina (Machine Learning).

Nuestra oferta de servicios consiste en asistir a empresas con sus canales Electrónicos con aplicaciones Web y Móvil para ofrecer servicios del mundo de la Especulación, Inversiones, Economía,

Finanzas y Riesgo. Así como el desarrollo de sistemas especializados para facilitar la gestión de inversiones, portafolios, trading, simuladores, modelos, plataformas de trading, finanzas corporativas, finanzas internacionales, valoración, optimización, riesgo, calculadoras, proyecciones, etc.

Iniciamos la conformación de un ecosistema para materializar esta oferta. En este sentido estamos buscando aliados que en medio de entornos complejos son capaces de identificar oportunidades y materializarlas.

Synergy Vision se encuentra desarrollando y actualizando nuevos materiales sobre temas como: Probabilidad y Estadística Matemática en R, Programación Científica en R, Mercados, Inversiones y Trading, Datos y Modelos Financieros en R, Renta Fija, Inmunización de Carteras de Renta Fija, Teoría de Riesgo en R, Finanzas Cuantitativas, Ingeniería Financiera, Procesos Estocásticos en R, Series de Tiempo en R, Ciencia de los Datos, Ciencia de los Datos Financieros, Simulación en R, Desarrollo de Aplicaciones Interactivas en R, Minería de Datos, Aprendizaje Estadístico, Estadística Multivariante, Riesgo de Crédito, Riesgo de Liquidez, Riesgo de Mercado, Riesgo Operacional, Riesgo de Cambio, Análisis Técnico, Inversión Visual, Finanzas, Finanzas Corporativas, Valoración, Teoría de Portafolio, entre otros.

Nuestra página web: <http://synergy.vision/nosotros/>
Nuestra cuenta de Twitter es (<https://twitter.com/bysynergyvision>) y nuestros repositorios están en GitHub (<https://github.com/synergyvision>). Somos Científicos de Datos.

Agradecimientos

Este libro es posible gracias a una gran cantidad de desarrolladores que contribuyen en la construcción de herramientas para generar

documentos enriquecidos e interactivos. En particular al autor de los paquetes Yihui Xie xie(2015).

A todo el equipo de Synergy Vision que no deja de soñar. Hay que hacer lo que pocos hacen, insistir, insistir hasta alcanzar. Lo más importante es concretar las ideas. La idea es sólo el inicio y solo vale cuando se concreta.

Synergy Vision, Caracas, Venezuela



1

Introducción

A diferencia del pasado cuando la información masiva era un lujo permitido solo a las grandes corporaciones públicas o privadas, hoy día gracias a la ubicuidad de las tecnologías digitales, casi cualquier persona o pequeño negocio puede, con poco esfuerzo, recibir ingentes cantidades de datos, sea a través del acceso a bases de datos, internet, comercio en línea, telefonía celular, redes sociales, sensores, transacciones comerciales, registros financieros, mensajería de texto, voz, audio y video, entre muchas otras. Además, los datos fluyen en una policromía de formatos análogos y digitales, como texto en papel o digital, imágenes, fotografías, archivos de audio y video, bases de datos, streaming, entre otras. A esta enorme cantidad de datos se le conoce como Big Data, y ahora tenemos el problema contrario, es decir cómo salir adelante en nuestros emprendimientos sin perdernos entre tanta información. Sin embargo es posible extraer pistas útiles de allí mediante un proceso conocido como minería de datos o data mining en inglés, y la tecnología clave para esta tarea es el machine learning o aprendizaje automático.

El machine learning es parte de la Ciencia de Datos o Data Science y fue creado para estudiar teorías, algoritmos y aplicaciones de sistemas que aprenden como los humanos. Su empleo es cada vez más amplio debido a que proporciona estrategias probadas para descubrir patrones entre miles o millones de datos. La extensión de su uso se debe a dos factores: el primero es la ya comentada accesibilidad a grandes volúmenes de información y el segundo, es la aparición de herramientas computacionales para manejarlos a costos cada vez más reducidos y con tecnologías asequibles, mediante softwares de código abierto como R y Python.

Los softwares de código abierto son publicados bajo una licencia de

código abierto o forman parte del dominio público. En el caso de R y Python pueden descargarse libremente y existe una comunidad de usuarios y desarrolladores que de manera gratuita y colaboradora, los mantienen actualizados y en constante mejora, con nuevas y cada vez más poderosas aplicaciones. De manera que el único obstáculo real para el uso del machine learning por parte del público de habla española de España y América Latina es su desconocimiento. Aparte de estudios académicos formales universitarios, existen excelentes recursos a los cuales acceder para el dominio del machine Learning. Sin embargo, la inmensa mayoría de estos últimos se encuentran en inglés o en otros idiomas. Además, suelen utilizar un lenguaje cargado de tecnicismos computacionales, matemáticos o estadísticos, que alejan a la mayoría, más interesada en resolver los urgentes problemas de su negocio o profesión, que en sumergirse en los oscuros detalles de un teorema. Por esta razón ofrecemos este texto en español a los empresarios, emprendedores, profesionales y estudiantes de pregrado y de postgrado de cualquier disciplina de España y América Latina, en una presentación que esperamos resulte especialmente útil, por su orientación práctica y libre de complicaciones excesivamente técnicas. En este libro se presentan ejemplos y algoritmos de varias técnicas de machine learning, que pueden ser comprendidos por cualquiera con un dominio matemático y estadístico básico, al nivel del primer o segundo año de la mayoría de las disciplinas del pregrado universitario, en España y América Latina. Por ello, esperamos que un público muy vasto pueda beneficiarse del contenido de este libro.

1.1. Clasificación de los Aprendizajes

Este libro se desarrolla siguiendo la visión estándar de dividir al machine learning en grandes clases o tipologías según el tipo de datos disponibles: **supervised learning** o **aprendizaje supervisado**, **unsupervised learning** o **aprendizaje no supervisado**, y **reinforcement learning** o **aprendizaje reforzado** o por re-

fuerzo. El aprendizaje supervisado sería el tipo más fundamental de machine learning, en el cual una computadora aprende de un usuario a través de preguntas y respuestas. La computadora aprende a vincular una pregunta con su respuesta a partir de muestras apareadas de preguntas y respuestas. El objetivo es que la computadora adquiera la capacidad de generalización, generando una respuesta apropiada a preguntas no aprendidas. De este modo, el usuario no tiene que enseñarle todo a la computadora, sino que ella puede hacer frente automáticamente a situaciones desconocidas aprendiendo sólo una fracción de conocimiento. El aprendizaje supervisado se ha aplicado con éxito a una amplia gama de temas, como el reconocimiento de escritura a mano, habla e imágenes, filtrado de spam, publicidad online, análisis de señales neuronales, análisis genético, predicción del precio de acciones, pronóstico del tiempo, entre muchos más. El aprendizaje supervisado es conocido como **regresión** si la respuesta es un valor real como la temperatura; **clasificación** o problema de reconocimiento de patrones, si la respuesta es un valor categórico como “sí” o “no”. Entre estos destacan los llamados **árboles de decisión** y los **random forest** o **bosques aleatorios**. Finalmente tenemos el **ranking** u **ordinal regression**, conocido en español como **ordenación**, si la respuesta es un valor ordinal como “bueno”, “normal” o “pobre”.

El aprendizaje no supervisado considera la situación donde no existe un usuario que actúe como orientador de la computadora, la cual recopila datos de forma autónoma a través de Internet u otra fuente de datos, e intenta extraer conocimientos útiles. El aprendizaje no supervisado es más automático que el aprendizaje supervisado, y su objetivo no necesariamente se especifica de manera clara. Entre las tareas típicas del aprendizaje no supervisado está la agrupación de datos y la detección de valores atípicos. Estas técnicas se utilizan con éxito en una amplia gama de problemas como el diagnóstico de sistemas, seguridad, detección de eventos y análisis de redes sociales. También se utiliza a menudo como un paso de preprocesamiento del aprendizaje supervisado. Entre las técnicas más conocidas se encuentran: **detección de outliers** y de **novedades**; **análisis de componentes principales** más conocido por sus siglas en inglés **PCA** de **Principal Components Analysis**; **clustering**; y

bosques aleatorios. El **clustering** o **agrupamiento** es una parte no supervisada de la clasificación, y su objetivo es categorizar las muestras de entrada en clústeres o grupos sin ninguna supervisión. Por lo general, se supone que muestras similares pertenecen al mismo grupo, y se supone que las muestras disímiles pertenecen a grupos diferentes. Por lo tanto, cómo medir la similitud entre las muestras es la cuestión clave en la agrupación. La detección de outliers o valores atípicos, tiene por objeto encontrar muestras irregulares en un conjunto de datos determinado. El objetivo de la detección de cambios o de novedad, es juzgar si un nuevo conjunto de datos tiene la misma propiedad que el conjunto de datos original.

El aprendizaje por refuerzo está orientado a adquirir la capacidad de generalización de la misma manera que en el aprendizaje supervisado, sin embargo el usuario que actúa como supervisor no responde directamente a las preguntas sino que evalúa el comportamiento del sistema y le proporciona retroalimentación al respecto. El objetivo del aprendizaje por refuerzo es que en base a la retroalimentación del supervisor, le permita a la computadora mejorar su comportamiento para maximizar el resultado de la evaluación. El aprendizaje por refuerzo es un modelo importante para el comportamiento de los humanos y los robots, y se ha aplicado a varias áreas, tales como el control autónomo de robots, juegos de computadora y estrategias de optimización de mercadeo. Detrás del aprendizaje por refuerzo, se utilizan a menudo los métodos de aprendizajes supervisados y no supervisados tales como como regresión, clasificación y agrupamiento. En este libro no abordaremos esta metodología.

1.2. Organización del texto

El libro se estructura en ocho capítulos, el **Capítulo 1** corresponde a la presente Introducción, mientras que en el **Capítulo 2** se inicia al lector en el uso de los softwares R y Phyton, mostrando como proceder a realizar su instalación dentro del ambiente de Anaconda. En todos los capítulos siguientes se sigue el mismo

orden de presentación del **Capítulo 2**, mostrando primero el uso del software R y luego el uso de Python para cada técnica estudiada.

El **Capítulo 3** se dedica a la lectura y preparación de los datos. En este capítulo introducimos al lector en la técnica de detección de valores atípicos, las cuales pueden ser objeto de aprendizajes supervisados, no supervisados o una combinación de los anteriores.

El **Capítulo 4** se dedica a la regresión lineal, inicio básico para todos los métodos de regresión. En el **Capítulo 5** analizamos la regresión logística, esencial en problemas con datos cualitativos o categóricos. Ambas técnicas de regresión corresponden a aprendizajes supervisados. En el **Capítulo 6** veremos una de las técnicas más usadas de aprendizaje no supervisado como es el análisis de clúster. Continuamos en el **Capítulo 7** con este tipo de aprendizaje, introduciendo las bases del análisis de componentes principales (PCA), utilizado como preprocesamiento para otras técnicas como la regresión lineal, porque permite reducir la dimensionalidad de las variables. En el **Capítulo 8**, investigamos métodos básicos basados en árboles de decisión incluyendo bosques aleatorios. Como parte integrante del texto se anexan los códigos en R y Python de cada práctica, los cuales pueden descargarse en XXXXXX

En el **Apéndice** se incluyen información adicional sobre los softwares utilizados y otros relacionados, mientras que las **Referencias** se presentan otras fuentes para aquellos interesados en continuar profundizando en el tema, aunque se tiene previsto la elaboración de otros recursos para avanzar en el conocimiento de la Ciencia de Datos.

1.3. Conjunto de Bases de Datos utilizadas

Algunas de las bases de datos utilizadas fueron creadas directamente para emplearlas en el libro. Otras pueden recuperarse fácilmente con ayuda de los códigos de R y Python, dado que son parte de las librerías de estos softwares o vienen por defecto con los mismos.

La Tabla 1 a continuación contiene una breve descripción de las mismas:

Tabla 1. Descripción de las bases de datos utilizadas en el libro

Nombre	Librería o ubicación	Descripción
Default	ISLR	Registros de clientes con incumplimiento de pagos de una empresa de tarjetas de crédito
EmployeeSalaries	https://data.montgomerycountymd.gov/api/views/2qd6-mr43/rows.csv?accessType=DOWNLOAD	Salarios de empleados del Condado Montgomery
IrisP	Distribución básica de R	Variedades morfológicas de la flor Iris de tres especies relacionadas
mtcars	Distribución básica de R	Consumo de combustible, caballos de fuerza y otra información sobre 32 automóviles. Son datos extraídos de la revista Motor Trend US de 1974
slsummary	http://www.fns.usda.gov/sites/default/files/pd/slsummary.xls	Participación y almuerzos servidos de 1969 a 2018 por el Programa Nacional de Almuerzo Escolar de EEUU
USArrests	Distribución básica de R	Estadísticas de delitos por cada 100.000 residentes en 50 estados de EE.UU.

2

Ambientes de trabajo para el científico de datos

Para usar las herramientas estadísticas y matemáticas, los científicos de datos usan lenguajes de programación que nos ahorran el trabajo de codificar paso a paso los algoritmos de los métodos a utilizar. La comunidad científica nos proporcionan estas soluciones a través de paquetes y librerías que podemos acceder generalmente de manera gratuita. El siguiente gráfico del portal web <https://www.paradigmadigital.com/dev/es-python-el-lenguaje-del-futuro/> muestra los lenguajes preferidos por los científicos de datos :

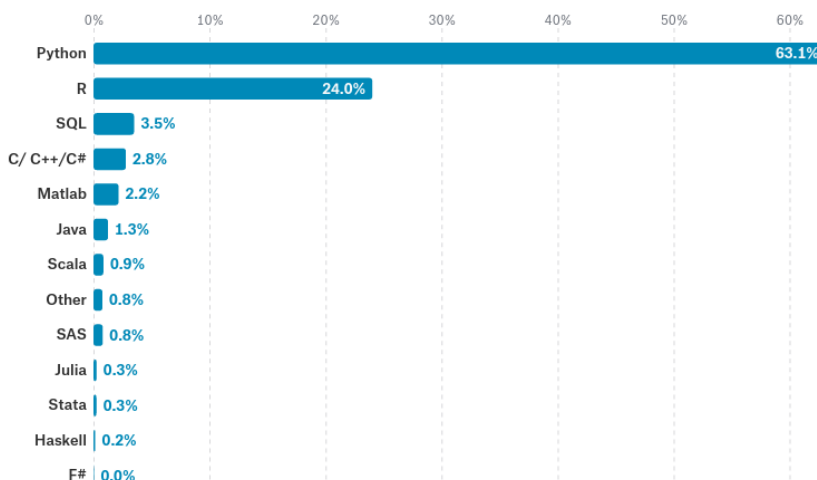


Figura 2.1

Sin duda *Python* y *R* son los favoritos. Ambos son de códigos abiertos (open source), Python es un lenguaje flexible fácil de

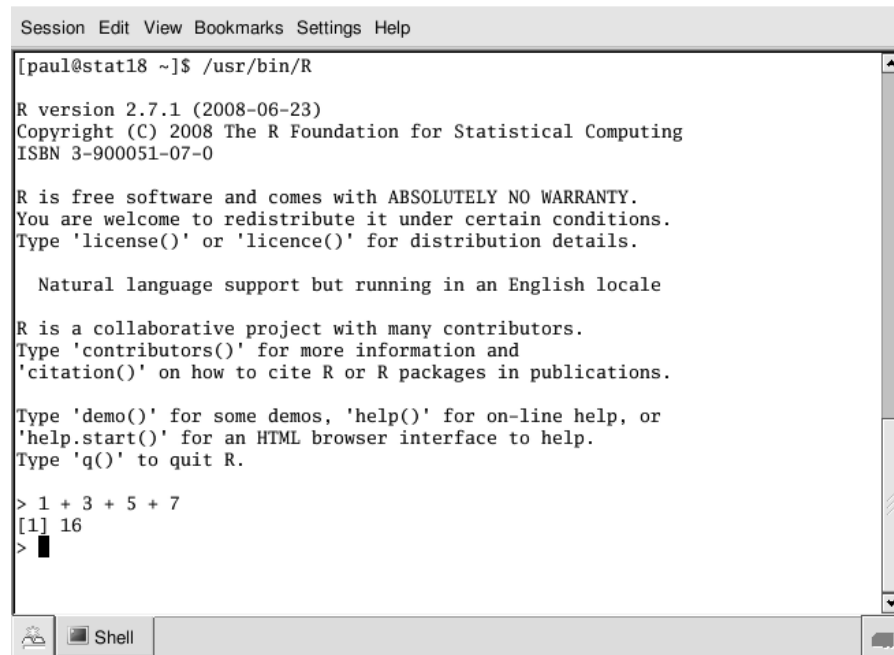
aprender y de propósito general, adaptable a fines empresariales. Por su parte, R es un lenguaje funcional orientado a la comunidad científica que trabaja con estadísticas, disponiendo de una infinidad de librerías, siempre en crecimiento. Entre ambos lenguajes Python es el más utilizado, sin embargo R puede adaptarse para fines empresariales gracias a librerías como Shiny, las cuales permiten crear páginas web donde se pueden aplicar prácticamente todas las herramientas de R y ofrecer resultados adecuados a todo tipo de usuarios.

En principio todo aquel que trabaje con datos debe conocer ambos lenguajes y utilizarlos de acuerdo a sus necesidades. En los ámbitos académicos, divulgativos o de investigación, es recomendable usar R. Python es una mejor opción si las necesidades son del campo empresarial, industrial o para el diseño de software en el cual se usen métodos estadísticos. Los dos lenguajes pueden descargarse en los siguientes enlaces:

- *R*: <https://www.r-project.org>
- *Python*: <https://www.python.org>

2.1. Primer encuentro con los lenguajes

Al descargar estos lenguajes lo primero que veremos es: en el caso de *R*:



```
Session Edit View Bookmarks Settings Help

[paul@stat18 ~]$ /usr/bin/R

R version 2.7.1 (2008-06-23)
Copyright (C) 2008 The R Foundation for Statistical Computing
ISBN 3-900051-07-0

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

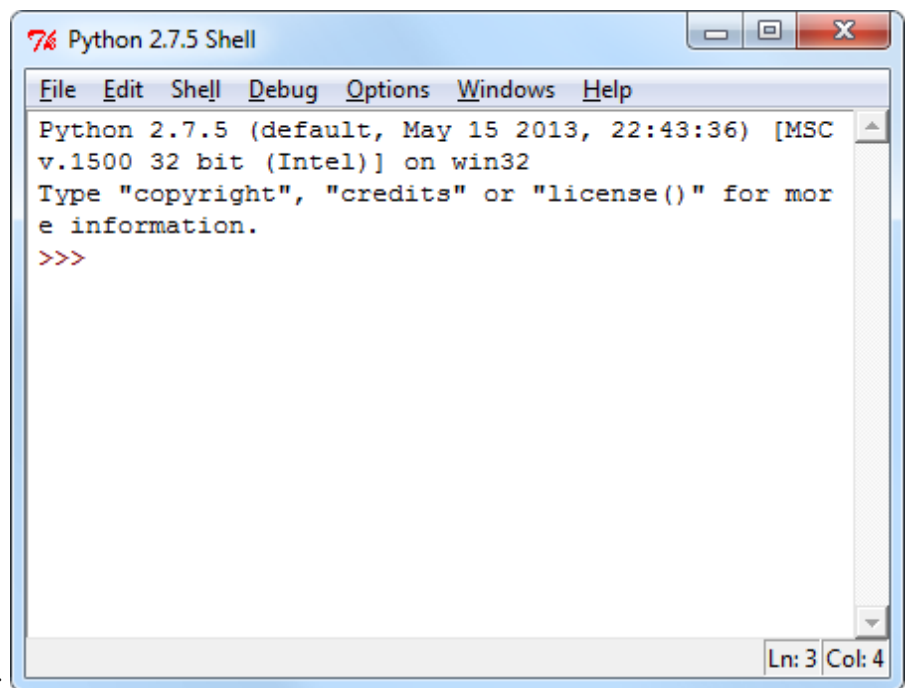
Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> 1 + 3 + 5 + 7
[1] 16
>
```

Figura 2.2



```
Python 2.7.5 Shell

File Edit Shell Debug Options Windows Help

Python 2.7.5 (default, May 15 2013, 22:43:36) [MSC
v.1500 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for mor
e information.
>>>
```

y en el caso de *Python*:

Aunque es posible comenzar a trabajar en estos ambientes, lo usual es elegir entornos más amigables.

Para R, el entorno más utilizado es RStudio el cual tiene la siguiente presentación:

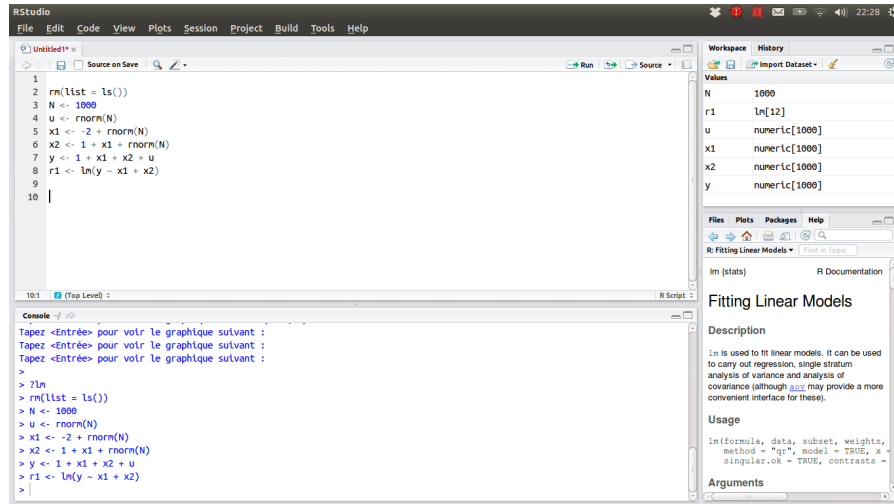


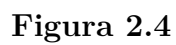
Figura 2.3

Para Python suele utilizarse Spyder y Jupyter Notebook. Las presentaciones de estos dos entornos son los siguientes:

Entorno de Spyder:

Entorno de Jupyter notebook:

Aunque con estos entornos se nos facilita mucho el trabajo seguimos con el inconveniente de que muchas de las librerías para la ciencia de los datos debemos instalarlas por nuestra cuenta, este problema se solventa con la existencia de ambientes de desarrollo para científicos de datos que incluyan todos estos factores.



2.2. Anaconda

Anaconda es una distribución libre y abierta que incluye los lenguajes *R* y *Python* que permite a los científicos de datos poder trabajar de una manera mas comoda y centralizada. Anaconda trae por defecto una gran cantidad de librerías orientadas a la ciencia de datos y proyectos de Machine Learning, ademas nos permite descargar *Rstudio* con un simple klik. La presentacion que veremos cuando abramos *Anaconda* es :

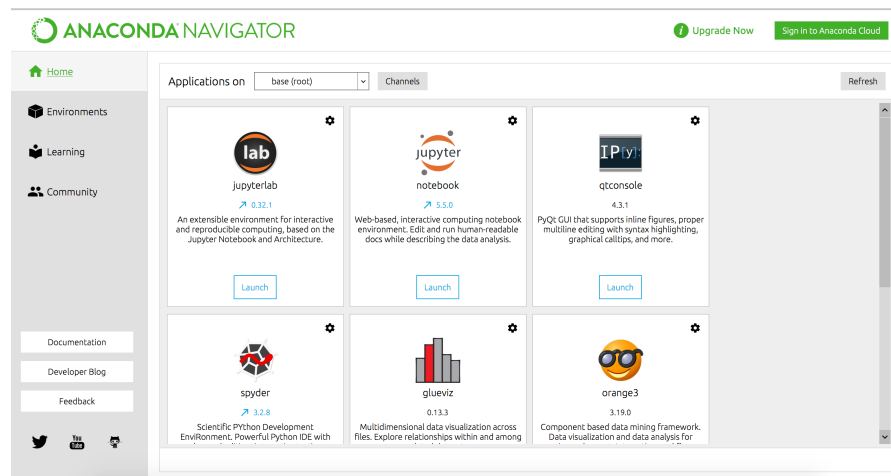


Figura 2.6

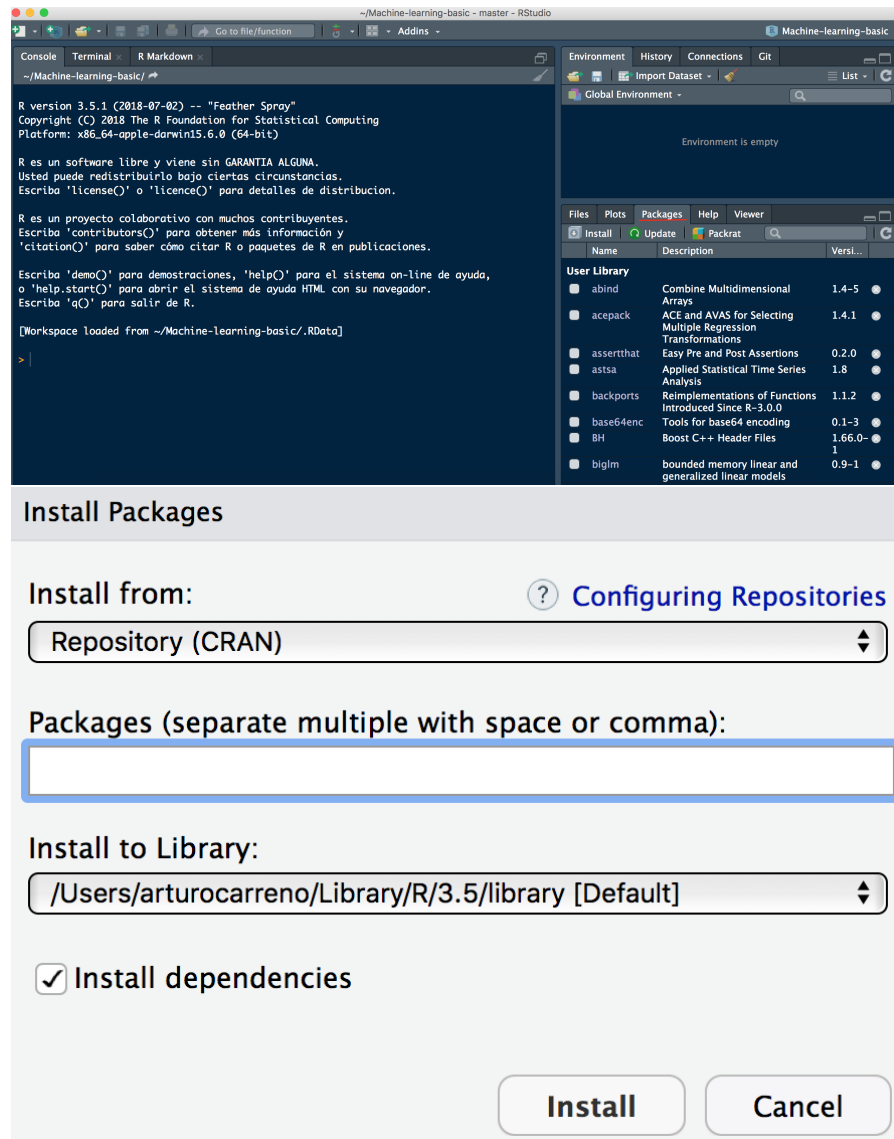
Se recomienda el uso de Anaconda para el público en general, para los principiantes debido a que permite un rapido inicio sin tener que descargar programas y paquetes por su propia cuenta y de esta forma concentrarse en el aprendizaje y para los avanzados ya que permite un trabajo mas centralizado y eficiente al poder tener un ambiente con todo los requerimientos que se necesiten para poder trabajar de una forma mas comoda. *Anaconda* se puede descargar en el siguiente enlace: <https://www.anaconda.com>

2.3. Librerías

A pesar de que al descargar Anaconda vienen por defecto una gran cantidad de librerías o paquetes, podríamos estar interesados en instalar alguno que tal vez no lo esté. En esta sección enseñaremos como instalar paquetes de forma rápida y sencilla.

2.3.1. Instalando librerías en *R*

Las descargas de las librerías en *R* la haremos directamente desde *RStudio*. Para ello hacemos clic en la pestaña *Package* ubicadas en la sección inferior derecha subrayada en rojo, luego haremos clic en el botón *Install*



Los campos de información solicitada se refieren a:

- *Install From*: Se utiliza para instalar paquetes desde la base de datos oficial de R o desde algún archivo específico. En este último caso es necesario indicar la ruta del archivo.
- *Package*: Debemos colocar el nombre del paquete que vamos a

instalar. Si deseamos instalar varios paquetes a la vez, debemos colocar sus nombres y separarlos por comas o puntos y comas.

- *Install to library*: Ruta donde estará guardado el paquete una vez instalado. Se recomienda dejar el valor por defecto.
- *Install dependencies*: Algunos paquetes necesitan del uso de otros paquetes para su correcto funcionamiento. Al tildar esta opción estamos otorgándole permiso a RStudio para que instale las librerías que necesita el paquete que deseamos instalar.

2.3.2. Instalando librerías en *Python*

Descargar librerías en *Python* no es tan fácil como en *R*, pero Anaconda facilita esta tarea. En el navegador de *Anaconda* vamos a la sección *Ambientes*, en inglés *Environments*, y al lado del buscador seleccionamos la acción *Todo*, *All* en inglés, colocamos el nombre del paquete que deseamos instalar y se nos desplegará en la parte inferior derecha dos opciones: *Aplicar* o *Apply* y *Borrar* o *Clear*. Seleccionamos la opción *Apply* y se instalará el paquete deseado.

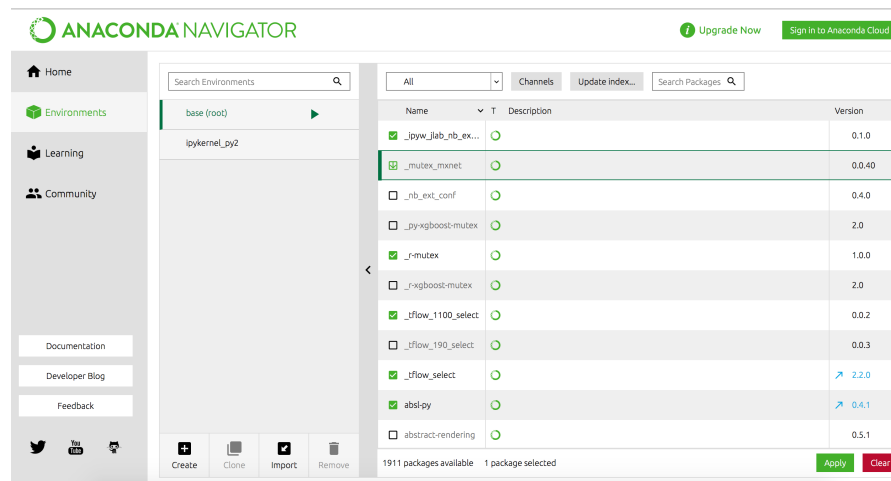


Figura 2.7

Si el paquete no está disponible en el ambiente tenemos la opción

de instalarlo desde la terminal de *Anaconda*. En la sección *Environment* hacemos clic en la flecha al lado de la palabra *base (root)* y seleccionamos *Open terminal*, tal como se muestra en la imagen

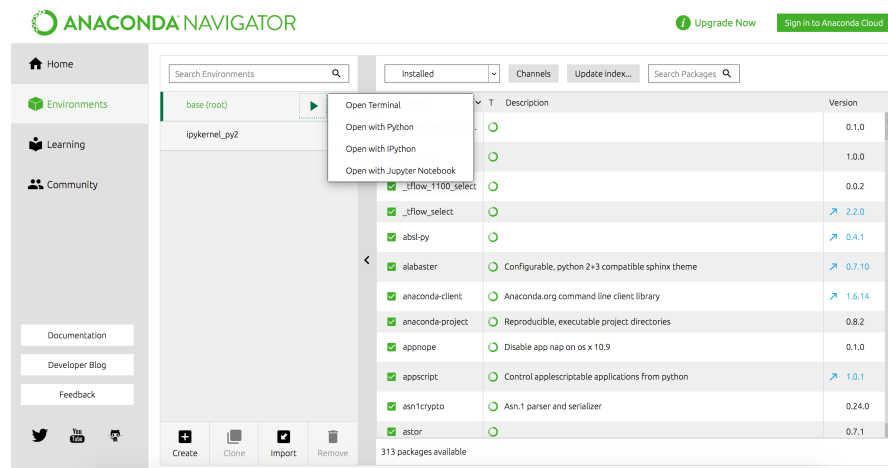


Figura 2.8

Se abre la consola del terminal y allí escribimos el comando *conda install* “Nombre del paquete” como se presenta a continuación

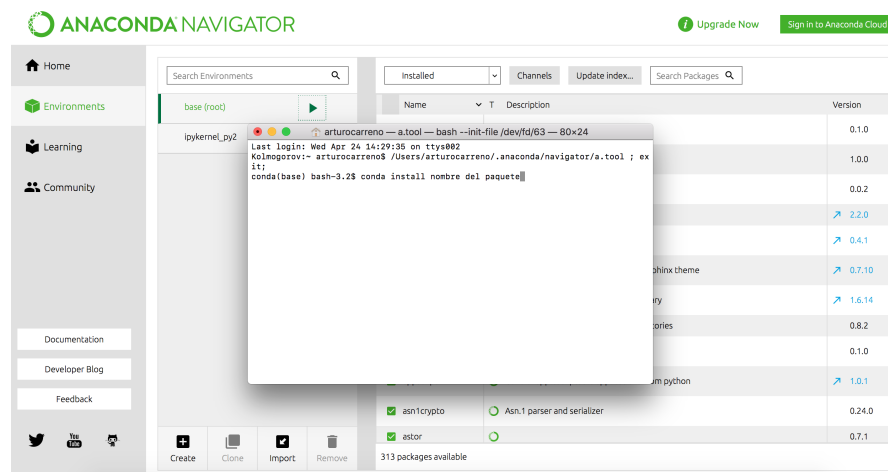


Figura 2.9

3

Uso y preparación de los datos

Al iniciar cualquier estudio debemos tener en cuenta que sin un buen conjunto de datos la mayoría de las técnicas que implementaremos darán resultados muy pobres. Si descuidamos la calidad de los datos, nuestros análisis se alejarán de la realidad que intentamos representar. La existencia de puntos atípicos, variables sin datos o con errores, pueden ser una verdadera pesadilla al iniciar nuestro estudio, por tal razón los lenguajes poseen herramientas que nos permiten manejar correctamente estas situaciones. En esta sección estudiaremos las herramientas básicas que *R* y *Phyton* ofrecen a todo analista que inicie un proyecto con datos con los cuales no esté familiarizado.

3.1. Lectura de datos

En general hay tres tipos de datos básicos con los cuales nos enfrentaremos, los cuales son archivos con extensiones *.csv*, *.txt*, *.xls*. Existen otras extensiones pero es muy raro que tenga que trabajar con datos provenientes de ellas, estas extensiones pueden ser: *.docx*, *.pdf*, entre otras

3.1.1. Lectura de datos con *R*

RStudio nos facilita mucho esta tarea, pues tiene una opción que nos permite cargar los datos y escoger la configuración del archivo sin tener que programar, veremos como hacer esto de forma directa

y luego de manera adicional presentaremos algunas funciones que hacen esta misma tarea.

- Archivos *csv* o *txt*: Primero buscamos en las opciones que se encuentran disponibles en la parte superior de *RStudio* y seleccionamos *archivo* o *file*, luego se desplegarán varias opciones de las cuales debemos seleccionar *Import Dataset* o *importar datos*, recomendamos utilizar la opción *From Text (readr)*..., debido a su alta variedad de opciones para cargar los datos.

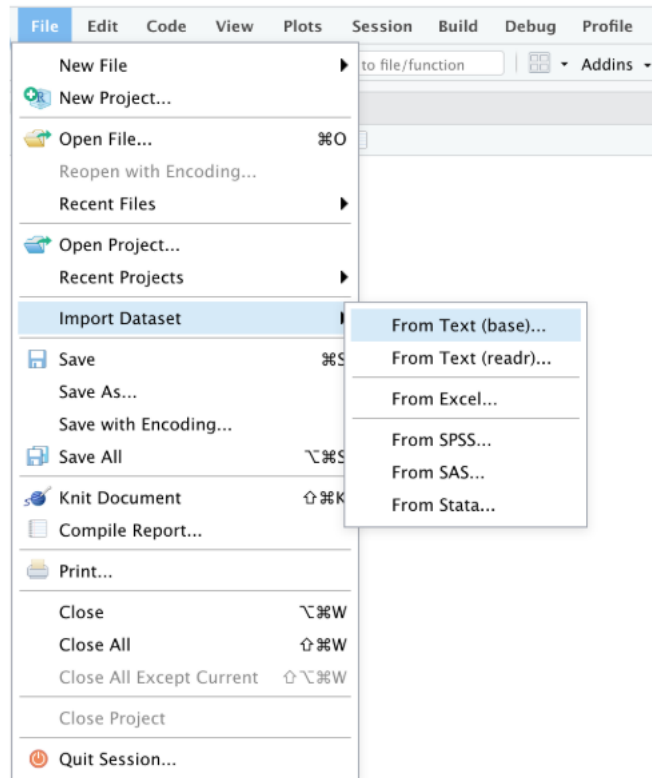


Figura 3.1

El paquete *readr* nos permite seleccionar las siguientes opciones

- (I) Cargar archivos directamente desde una ruta de nuestro computador o de una dirección web.

- (II) Seleccionar con un simple clic el tipo de datos de alguna columna en particular.
- (III) Mantener o eliminar columnas.
- (IV) Cambiar el nombre al conjunto de datos.
- (V) No usar una cantidad N de las primeras filas del conjunto de datos.
- (VI) Usar el encabezado como nombre de las columnas
- (VII) Recortar espacios en los nombres
- (VIII) Cambiar los delimitadores de las columnas (separadas por comas, puntos y comas, tabulaciones, guiones, etc).
- (IX) Seleccionar la codificación de los datos.
- (X) Identificar de alguna forma en particular los valores Na

Import Text Data

File/URL:

Data Preview:

Full Name (character)	Gender (character)	Current Annual Salary (double)	2017 Gross Pay Received (double)	2017 Overtime Pay (double)	Department (character)	Department Name (character)	Division
Aarhus, Pam J.	F	70959.79	71316.72	0.00	POL	Department of Police	MSB Information Mgmt and Tech Division Rec
Aaron, Marsha M.	F	110359.00	108040.82	0.00	HHS	Department of Health and Human Services	Adult Protective and Case Management Serv
Abadio, Godfred A.	M	55950.24	62575.19	7649.19	COR	Correction and Rehabilitation	PRRS Facility and Security
Ababu, Essayas	M	95740.00	96055.94	0.00	HCA	Department of Housing and Community Affairs	Affordable Housing Programs
Abbamonte, Drew B.	M	74732.00	98736.78	23468.73	POL	Department of Police	PSB 6th District Special Assignment Team
Abbasian, Takin M.	M	16451.50	4547.10	94.92	POL	Department of Police	MSB Personnel Division
Abdalla, Eiman M.	F	63977.00	62177.20	184.56	FIN	Department of Finance	General Accounting
Abdelhamed, Shereen N.	F	53274.00	50549.01	1872.92	POL	Department of Police	PSB 2nd District Patrol
Abdelmoniem, Marwan M.	M	68739.36	65719.27	0.00	HHS	Department of Health and Human Services	Head Start
Abdool, Daniel Z.	M	50172.00	48567.94	1384.89	FRS	Fire and Rescue Services	Field Recruits
Ahfulu-Ghani, Maxinah I.	F	50576.93	51657.55	7993.30	PRR	Department of Police	FSR Traffic Division Automated Traffic Infrastr

Import Options:

Name: ☒ First Row as Names Delimiter: Escape:

Skip: ☒ Trim Spaces Quotes: Comment:

☒ Open Data Viewer Locale: NA:

Code Preview:

```
library(readr)
rows <- read_csv("https://data.montgomerycountymd.gov/api/views/2qdg6-mr43/rows.csv?accessType=DOWNLOAD")
View(rows)
```

Figura 3.2

- Archivos *.xls* (*MS-Excel*): Para cargar archivos *.xls* debemos seleccionar la opción *From Excel* que se encuentra en la misma ubicación de las anteriores. Ésta opción usa el paquete *readxl*. La ventaja de este paquete es que posee prácticamente las mismas opciones que los paquetes de *R* básico para importar archivos de

Excel, pero además permite seleccionar la hoja de cálculo a usar del documento *Excel*. La siguiente imagen representa lo que en general encontraremos al cargar un archivo *Excel*:

The screenshot shows the 'Import Excel Data' dialog box. At the top, there's a 'File/Url:' field with the URL 'http://www.fns.usda.gov/sites/default/files/pd/slsummar.xls' and an 'Update' button. Below this is a 'Data Preview:' section showing a table with columns X_1 through X_6. The table contains data for 'NATIONAL SCHOOL LUNCH PROGRAM PARTICIPATION AND LUNCHES SERVED' for the year 1969. The 'Import Options:' section at the bottom includes fields for 'Name' (slsummar), 'Max Rows' (empty), 'Sheet' (Default), 'Range' (A1:D10), and checkboxes for 'First Row as Names' and 'Open Data Viewer'. A 'Code Preview:' section shows the R code for importing the data using the *readxl* package.

	X_1 (character)	X_2 (character)	X_3 (character)	X_4 (character)	X_5 (character)	X_6 (character)
(Data as of October 05, 2018)	NA	NA	NA	NA	NA	NA
-----Average Participation-----	NA	NA	NA	NA	NA	NA
Fiscal	Free	Reduced	Full	Total Lunches	Percent Free/RP	
Year	Millions	Price	Price	Served	of Total	%
1969	2.8999999999999999	11	16.5	19.399999999999999	3368.1999999999998	15.1
1970	4.5999999999999996	11	17.800000000000001	22.399999999999999	3565.0999999999999	20.699999999999999
1971	5.7999999999999998	0.5	17.800000000000001	24.100000000000001	3848.3000000000002	26.100000000000001
1972	7.2999999999999998	0.5	16.600000000000001	24.399999999999999	3972.0999999999999	32.399999999999999

Figura 3.3

Gracias a *readxl* podemos simplificar y mejorar los datos, en la variable *skip* se coloca un valor numérico para indicar cuantas de las primera líneas se deben eliminar. Dando clic derecho sobre la variable número 3 cambiamos el tipo que trae por defecto. A continuación se muestra la imagen.

La importación de datos CSV o TXT puede hacerse directamente desde el R básico con paquetes especialmente diseñados para esta tarea como se muestra en el Apéndice.

3.1.2. Lectura de datos con *Python*

La variedad de propósitos para los cuales se usa el lenguaje Python ha conducido a la existencia de una gran cantidad de formas de leer archivos *.csv*, *.txt*, *.xlsx*. Sin embargo la gran mayoría de los especialistas en Machine Learning que utiliza este lenguaje, optan por la librería *Pandas* para la manipulación de datos. Por esta razón usaremos la funciones que nos ofrece *Pandas*: *pandas.read_csv()* y *pandas.read_excel()*. Ambas funciones aceptan una gran cantidad

Import Excel Data

File/Url:

Data Preview:

X_1 (numeric)	X_2 (double)	X_3 (numeric)	X_4 (double)	X_5 (double)	X_6 (double)	X_7 (numeric)
1969	2.9	NA	16.5	19.4	3368.20	15.1
1970	4.6	NA	17.8	22.4	3565.10	20.7
1971	5.8	0.5	17.8	24.1	3848.30	26.1
1972	7.3	0.5	16.6	24.4	3972.10	32.4
1973	8.1	0.5	16.1	24.7	4008.80	35.0
1974	8.6	0.5	15.5	24.6	3981.60	37.1
1975	9.4	0.6	14.9	24.9	4063.00	40.3
1976	10.2	0.8	14.6	25.6	4147.90	43.1
1977	10.5	1.3	14.5	26.2	4250.00	44.8
1978	10.3	1.5	14.9	26.7	4294.10	44.4
1979	10.0	1.7	15.3	27.0	4357.40	43.6
1980	10.0	1.9	14.7	26.6	4387.00	45.1

Previewing first 50 entries.

Import Options:

Name: Max Rows: ☐ First Row as Names

Sheet: Skip: ☒ Open Data Viewer

Range: NA:

Code Preview:

```
library(readxl)
url <- "http://www.fns.usda.gov/sites/default/files/pd/slsummar.xls"
destfile <- "slsummar.xls"
curl::curl_download(url, destfile)
slsummar <- read_excel(destfile, col_names = FALSE,
  col_types = c("numeric", "numeric", "numeric",
```

Figura 3.4

de parámetros, pero solo mencionaremos los más importantes. Una información mas detallada sobre estas funciones pueden encontrarse en:

- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html
- https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_excel.html

A diferencia de *RStudio* no hay una forma en común para cargar los archivos, sin que tengamos que escribir algunas líneas de código. A continuación mostraremos como hacerlo:

- `filepath_or_buffer`: Ruta del archivo o dirección web.
- `Sep`: valor que separa los datos, por defecto es una coma
- `header`: Se usa para identificar en que posición se encuentra la fila que será usada como los nombres de las columnas.
- `names`: Lista con el nombre de las columnas a usar.
- `index_col`: Columna que será usada por defecto como índice

- *decimal*: Carácter que será usado como carácter decimal, por defecto es un punto
- *usecol*: Este parámetro nos permite decidir qué número de columnas usar.
- *dtype*: Tipo de dato de cada columna, por ejemplo {'a': np.float64, 'b': np.int32, 'c': 'Int64'}.

```
import numpy
import pandas

g = pandas.read_csv("train.txt", sep=",", header=0, decimal=".",
                    index_col=0, dtype={"target": numpy.float64})

g.head()
```

	target	0	1	2	3	4	5	6	7	8	...	290	291	292	293	294
id																
0	1.0	-0.098	2.165	0.681	-0.614	1.309	-0.455	-0.236	0.276	-2.246	...	0.867	1.347	0.504	-0.649	0.672
1	0.0	1.081	-0.973	-0.383	0.326	-0.428	0.317	1.172	0.352	0.004	...	-0.165	-1.695	-1.257	1.359	-0.808
2	1.0	-0.523	-0.089	-0.348	0.148	-0.022	0.404	-0.023	-0.172	0.137	...	0.013	0.263	-1.222	0.726	1.444
3	1.0	0.067	-0.021	0.392	-1.637	-0.446	-0.725	-1.035	0.834	0.503	...	-0.404	0.640	-0.595	-0.966	0.900
4	1.0	2.347	-0.831	0.511	-0.021	1.225	1.594	0.585	1.509	-0.012	...	0.898	0.134	2.415	-0.996	-1.006

Figura 3.5

- (II) Leer archivos *.xlsx* con *pandas.read_excel()*: Usa prácticamente los mismos parámetros que la función anterior, con la diferencia de que no acepta el parámetro *sep* y usa un parámetro adicional llamado *sheet_name*, este parámetro debe ser una lista con los números de columnas a usar o con los nombres de dichas columnas.

3.2. Tratamiento previo de los datos

Luego de cargar los datos puede empezar la verdadera pesadilla para aplicar los métodos, debido a factores como errores de medición, problemas de codificación y errores humanos. En general clasificaremos estos problemas en dos grupos: *Valores Faltantes* o

Missing Values y *Valores Atípico* más conocidos por el término en inglés *Outliers*.

3.2.1. Missing Values o valores faltantes

Estos valores suelen generarse por diversas razones, entre las cuales se encuentran: problemas de codificación con los datos, errores de medición y errores humanos, entre otros. En algunos casos pueden generarse por problemas con la función que esté encargada de leer los datos, sea porque no lee bien los parámetros o simplemente por incompatibilidad del archivo. En esta sección veremos cuáles son las técnicas más usuales para solucionar estos problemas.

- (I) Missing values en *R*: Para trabajar con datos faltantes en *R* lo primero es contarlos. Para ello usaremos la función *sapply()* la cual permite aplicar funciones a columnas de manera explícita. En nuestro caso aplicaremos la función *is.na()* la cual cuenta la cantidad de valores faltantes por columnas, la función *sum()* que suma todos estos valores y nos da el resultado final, y la función *dim()* la cual nos da las dimensiones de los datos.

```
library(readr)

train<-read_csv("train.txt", col_types=cols(id=col_skip()))

missing_values_count=sapply(train, function(train) sum(is.na(train)))

missing_values_total=sum(missing_values_count)
Total=dim(train)[1]*dim(train) [2]
missing_values_total/Total
```

Figura 3.6

En general, los valores faltantes tienden a eliminarse, pero esto no se puede hacer a la ligera, en otros casos suelen sustituirse por un valor común del conjunto de datos (esto se hace cuando tenemos distintas entradas para un individuo y alguna característica está vacía). También se debe chequear si la distribución de datos faltantes es

aleatoria. Sería muy extraño que cada 100 datos exactamente exista un dato faltante en un lugar en específico. Este último tipo de comportamiento hace pensar en que estamos en presencia errores sistemáticos.

Como tratar estos casos, dependerá de nuestros propósitos específicos, pues los datos faltantes tienden a sesgar de manera considerable muchas técnicas estadísticas. Cuando elegimos no usarlos decimos que los eliminaremos y cuando optamos por sustituirlos por un valor en específico diremos que los imputaremos. Para eliminar los datos faltantes sin riesgo a perder información vital, lo primero que debemos asegurarnos es que la proporción de estos datos con el total sea muy bajo, por ejemplo 300:1, si los valores faltantes representaran el 1 % de la población no es muy recomendable que los eliminemos. Un ejemplo típico es que pudieran representar un segmento de la población que por problemas de medición o de transmisión de los datos, simplemente no se pudo completar el llenado de información y el eliminarlos pudiera dejar por fuera una parte importante de la población. El 1 % no parece significativo, pero si tenemos 3 millones de datos el 1 % representaría 30 mil, una cantidad considerable, de posibles consumidores para un producto en específico.

Para eliminar los valores faltantes de la data por filas, usamos la función *na.omit()*, si queremos eliminar por columnas usamos la función *t()* la cual nos traspone los datos, el problema es que modificará el índice. Solventamos este problema con la función *rownames()*. Otro detalle es que la función *na.omit()* cambia el formato de los datos, por lo que usaremos la función *as.data.frame()* para corregir el problema.

- Sustitución por la media: Cuando los datos poseen un comportamiento uniforme alrededor de una región del plano se suelen sustituir por el valor medio que presentan los datos.

Para sustituir por el valor medio se utiliza la función *mean()* para calcular el valor medio de la columna y la función *replace()* para realizar el cambio.

- Sustitución por la mediana: El problema de sustituir por la

```
#### Por filas  
  
d = na.omit(train)  
class(d)  
d = as.data.frame(d)  
  
#### Por Columnas  
  
e = t(train)  
e = na.omit(e)  
e = as.data.frame(e)  
e = t(e)  
rownames(e) <- 1:nrow(e)
```

Figura 3.7

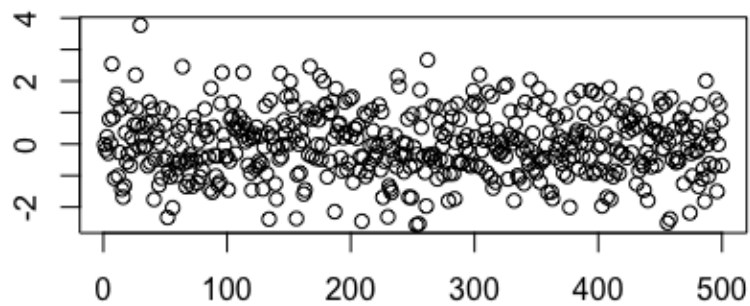


Figura 3.8

```
media = mean(train$'0')  
  
train$'0'[is.na(train$'0')] <- media
```

Figura 3.9

media es que ésta se puede ver afectada por valores atípicos (los veremos en la próxima sección de este capítulo), es por esto que debe sustituirse por la mediana que es un estimador más robusto para este propósito. Por ejemplo, el conjunto de puntos 1,2,3,1,2,1,3,1,2,1,2,1,33 tiene promedio 4, sustituir por este valor no tiene sentido, mientras que la mediana es 2. Para realizar el cambio usamos *median()*

```
mediana = median(train$'0')
train$'0'[is.na(train$'0')] <- mediana
```

Figura 3.10

- Sustitución por la moda: La sustitución por la moda es especialmente útil cuando estamos en presencia de datos categóricos, recordemos que la moda no es mas que el dato que más se repite en el conjunto de datos. Para realizar el cambio usamos *mode()*

```
library(modeest)
moda = mfv(train$'0')
train$'0'[is.na(train$'0')] <- moda
```

Figura 3.11

- (II) Missing values en *Python*: Para identificar valores en *Python* a través de la librería Pandas existen diversas funciones que nos ayudarán. Lo primero que debemos hacer es contar la cantidad de datos faltantes, esto lo hacemos con las funciones *isnull()* y *sum()* de la librería Pandas. El resultado final será el número de missing values por columnas, la primera función los identifica y la segunda función los cuenta. La siguiente imagen muestra cómo hacerlo.

Ahora si decidimos eliminar los datos, realizar esto en *Python* es sencillo. Primero calculamos el porcentaje de datos faltantes

```
train = pandas.read_csv("train.txt", sep=",", header=0, decimal=".",
                        index_col=0, dtype={"target": numpy.float64})
```

```
missing_values_count = train.isnull().sum()
```

```
missing_values_count
```

```
target    0
0         0
1         0
2         0
3         0
4         0
5         0
6         0
7         0
8         0
9         0
10        0
11        0
12        0
13        0
14        0
15        0
16        0
17        0
18        0
19        0
..        ..
```

Figura 3.12

usando las funciones *pandas.shape()*, *pandas.product()*. La primera nos indica la dimensión del dataframe y la segunda realiza la multiplicación para poder saber la cantidad de celdas existentes.

```
missing_values_count = train.isnull().sum()
```

```
total_cells = numpy.product(train.shape)
total_missing = missing_values_count.sum()
total_missing/total_cells*100
```

Figura 3.13

Luego, dependiendo como estén organizados los datos, es decir, registros por columnas o por filas, sabremos como eliminar los registros con valores faltantes. En la función *df.dropna()*, si colocamos como parámetros *axis=0* se eliminarán los registros por fila que contengan valores faltantes y *axis=1* por columnas.

```
Columns_with_na_dropped = train.dropna(axis=1)
```

Figura 3.14

La otra alternativa con los datos faltantes es imputarlos. Para

ello seguiremos el mismo procedimiento usado con R , es decir imputación por la media, mediana y moda.

- Sustitución por la media: Para sustituir por el valor medio la función `.mean()` para calcular el valor medio de la columna y la función `.replace()` para realizar el cambio.

```
train.head()
```

	target	0	1	2	3	4	5	6	7	8	...	290	291	292	293	294
id																
0	1.0	2.000	2.165	0.681	-0.614	1.309	-0.455	-0.236	0.276	-2.246	...	0.867	1.347	0.504	-0.649	0.672
1	0.0	1.081	-0.973	-0.383	0.326	-0.428	0.317	1.172	0.352	0.004	...	-0.165	-1.695	-1.257	1.359	-0.808
2	1.0	-0.523	-0.089	-0.348	0.148	-0.022	0.404	-0.023	-0.172	0.137	...	0.013	0.263	-1.222	0.726	1.444
3	1.0	0.067	-0.021	0.392	-1.637	-0.446	-0.725	-1.035	0.834	0.503	...	-0.404	0.640	-0.595	-0.966	0.900
4	1.0	2.347	-0.831	0.511	-0.021	1.225	1.594	0.585	1.509	-0.012	...	0.898	0.134	2.415	-0.996	-1.006

5 rows × 301 columns

```
promedio = train["0"].mean()
train["0"] = train["0"].replace(numpy.nan, promedio)
```

Figura 3.15

- Sustitución por la mediana: Para realizar el cambio usamos `.median()`

```
mediana = train["0"].median()
train["0"] = train["0"].replace(numpy.nan, mediana)
```

Figura 3.16

- Sustitución por la moda: Para realizar el cambio usamos `.mode()`

```
moda = train["0"].mode()
train["0"] = train["0"].replace(numpy.nan, moda)
```

Figura 3.17

3.3. Identificación y tratamientos de Outliers o puntos atípicos

Los puntos u observaciones atípicas como su nombre lo indica son observaciones que tienen un comportamiento distinto al de las demás observaciones, por ejemplo en el conjunto de datos 2, 3, 2, 4, 3, 2, 34, 3, 2, 1, 2 el valor 34 es un punto atípico. Existen varias razones fundamentales para lo cual es vital detectar estas observaciones entre las que podemos mencionar están:

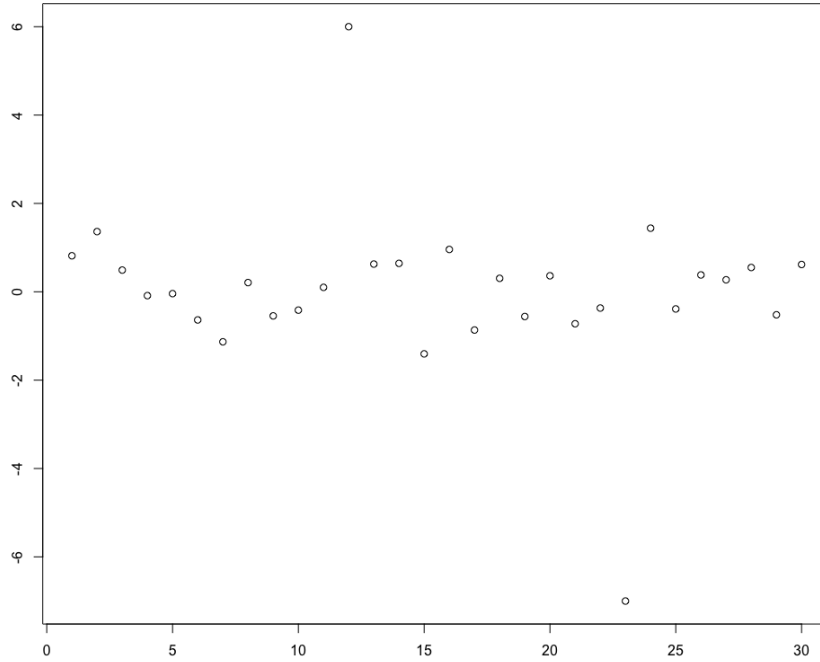
- Detectar errores en los datos: Si estamos observando un conjunto de datos donde en una columna denotamos la edad de los individuos y un individuo tiene en esa entrada -5 o 335 claramente existe un error en los datos
- Encontrar un subconjunto distinto dentro del conjunto original: En algunos casos los valores atípicos señalan la existencia de individuos en particular, cuya existencia puede ser importantes en nuestros estudios.
- Para evitar estimaciones incorrectas en algunos modelos estadísticos: Algunos modelos estadísticos tienen fuertes restricciones en las hipótesis, es por esto que la eliminación de algunos puntos atípicos puede mejorar de manera significativa nuestros resultados.

3.3.1. Identificación de puntos atípicos

Estudiaremos los dos enfoques más populares: el visual y el cuantitativo

- Visual: Como su nombre lo indica consiste en analizar los datos de manera visual para identificar los valores atípicos. Por ejemplo, en la siguiente imagen podremos notar claramente un par de valores atípicos:

El problema se complica cuando aumenta la dimensionalidad del conjunto de datos, es decir, aumentan las categorías que poseen las

**Figura 3.18**

observaciones. Por ejemplo, supongamos que tenemos un conjunto de datos con tres categorías x_1 , x_2 , x_3 , x_4 , en este caso lo usual es graficar el cruce de las variables para encontrar valores atípicos. El problema es que algunos cruces pueden no mostrar nada anormal, como es el cruce entre las variables x_1 y x_2 .

Pero al observar el cruce ente las variables x_1 y x_3 se puede detectar rápidamente el punto atípico.

Ahora, es claro que este método puede ser muy ineficiente a la hora de trabajar con una gran cantidad de atributos, es por eso que existe la necesidad de usar algún método cuantitativo para medir si un dato es atípico o no. Uno de los más utilizado es el método basado en la medida o distancia D_M de *Mahalanobis*

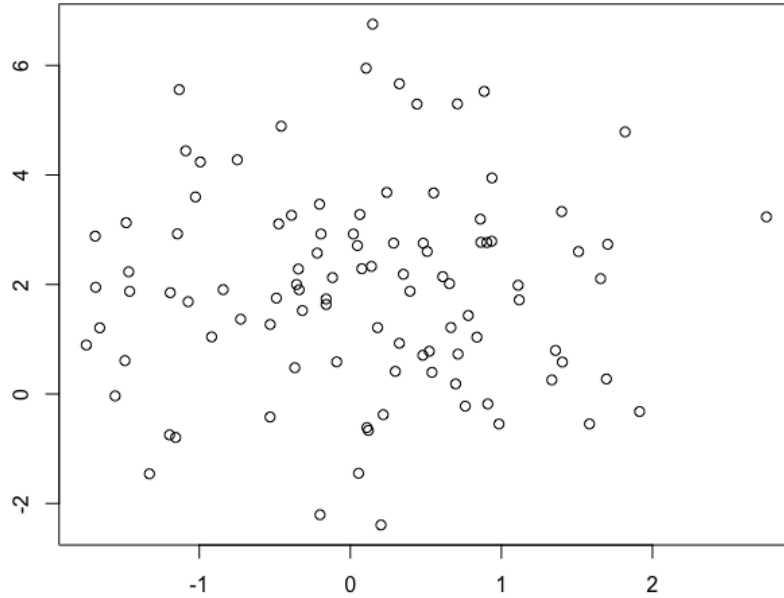


Figura 3.19

- D_M de Mahalanobis: Es una medida de distancia introducida por *Prasanta Chandra Mahalanobis* y su principal uso era determinar la similitud entre dos variables aleatorias. Su gran característica es que usa la matriz de covarianza para dar un peso ponderado por esta matriz a cada variable. Su gran desventaja es que es sensible a las escalas de los datos, por lo que se sugiere escalar los datos ya sea por su media u otro criterio similar.

En general, debemos tener un punto de referencia para poder medir la proximidad de los puntos que estamos midiendo, lo común es usar la media o la mediana, la ecuación que representa la distancia D_M de Mahalanobis es:

$$\sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

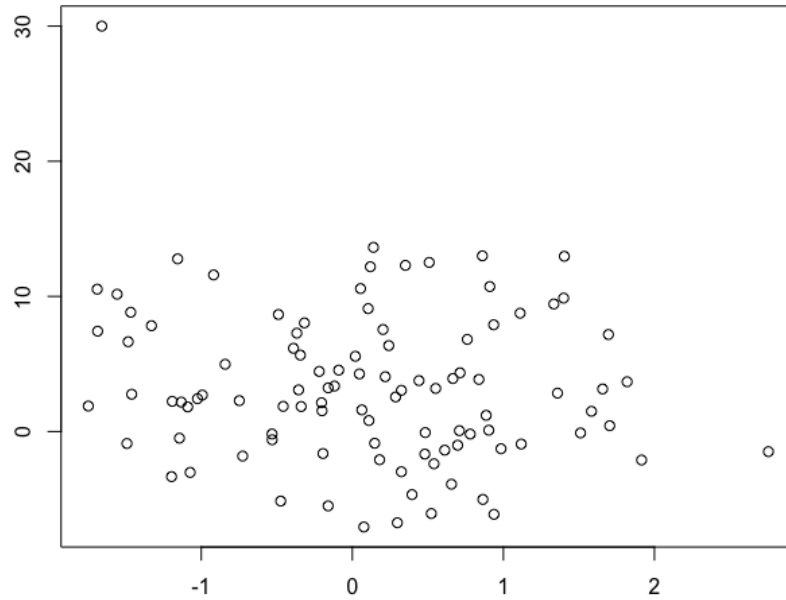


Figura 3.20

Donde y es el punto fijo de referencia, x es el punto al que le estamos calculando la distancia y Σ la matriz de covarianza.

3.3.2. Detección de outliers en R

- Visual: Supongamos que tenemos el conjunto Data previamente cargado y compuesto por tres variables x_1 , x_2 y x_3 . Usaremos la función `plot()`, para observar la variable x_1 . Usamos el siguiente comando:

Dando como resultado:

A primera vista no hay ningún valor atípico, sin embargo, si gene-

```
plot(data$x1, xlab="Variable x1", ylab="Rango", main="Única variable")
```

Figura 3.21

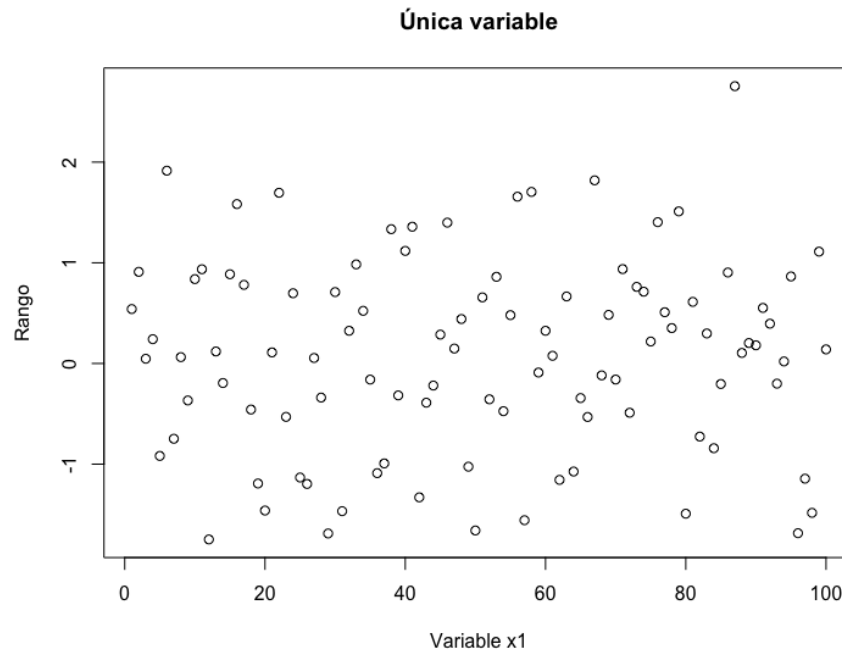


Figura 3.22

ramos el mismo gráfico para la variable x_3 , obtenemos claramente un outlier:

Ahora si queremos hacer el cruce de variables, usamos igual el mismo comando, por ejemplo, el de las variables x_1 y x_2 , hacemos:

Aparentemente no se observan outliers, como muestra la siguiente imagen.

Ahora, si realizamos el cruce de las variables x_1 y x_3 obtenemos:

Observando claramente un outlier, que posee un valor de 30. Ahora

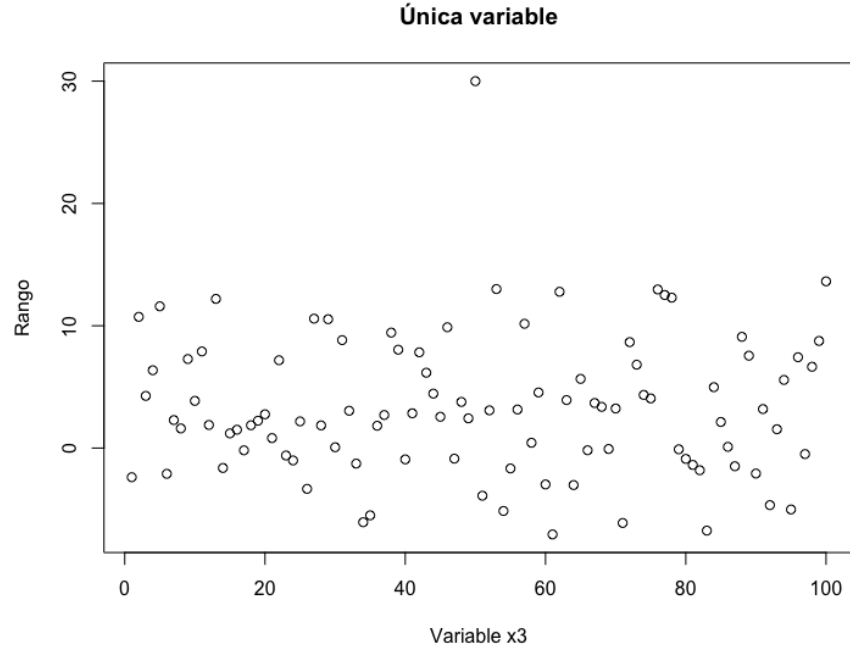


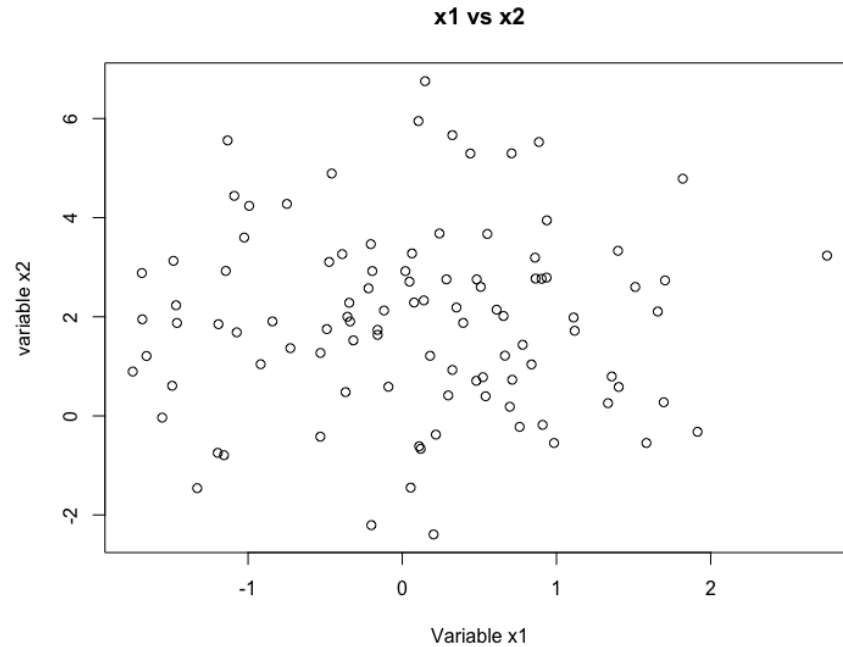
Figura 3.23

```
plot(data$x1, data$x2, xlab="Variable x1", ylab="variable x2", main="x1 vs x2")
```

Figura 3.24

si tenemos muchas variables, este proceso se complica, es por esto que usaremos la distancia D_M de *Mahalanobis*.

- la distancia D_M de *Mahalanobis*: Para calcular la distancia de Mahalanobis debemos calcular primero el punto de referencia, en nuestro caso la media, usaremos la función `apply()`, la función `cov()` para calcular la matriz de covarianza y al final usamos la función `mahalanobis()` indicándole los datos a calcular. Luego usamos la función `plot()` para ver la existencia de valores atípicos.

**Figura 3.25**

3.3.3. Detección de outliers en Python

- Visual: Para identificar outliers de manera visual en *Python* haremos uso de la librería *Matplotlib*, la cual nos brinda un sinfín de herramientas para poder llevar a cabo nuestros objetivos. Al igual que en la sección anterior usaremos el conjunto de datos Data con tres variables.

Para ver la gráfica de la variable x_1 , primero debemos seleccionar la variable, esto lo hacemos con el método `.iloc()` de *Pandas* el cual nos permite seleccionar subconjuntos de datos. Luego, usamos `plt.plot()` para generar la gráfica de los datos, el parámetro `ro` indica el tipo de gráfico, en nuestro caso en punteado; `plt.xlabel` para dar título al eje horizontal y `plt.ylabel` para dar título al eje vertical, finalmente usamos `plt.show` para mostrar la imagen.

Al graficar la variable x_1 , se observa que carece de valores atípicos:

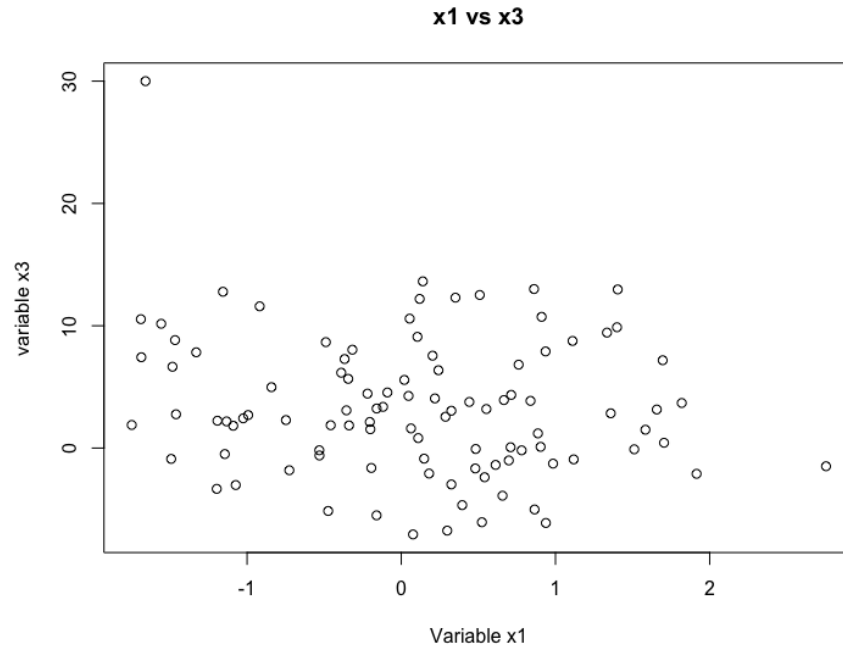


Figura 3.26

```
mahalanobis <- mahalanobis(data,center = apply(data, 2 ,mean), cov(data))
plot(mahalanobis, xlab="Mahalanobis", ylab="Rango",main=" ")
```

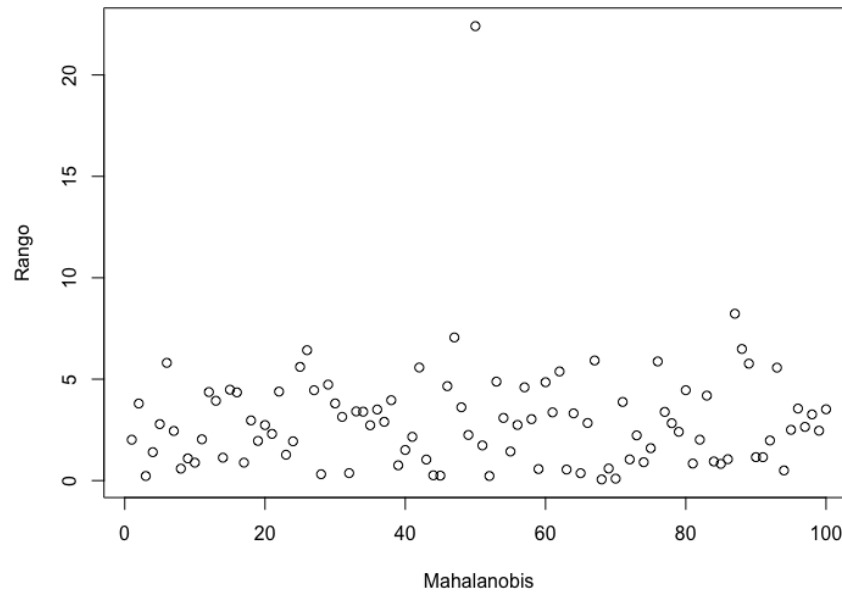
Figura 3.27

Pero cuando generamos el gráfico de la variable x_3 notamos el valor atípico:

Para realizar el gráfico del cruce de variables, simplemente agregamos la variable adicional como parámetro en `plt.plot()`. En este caso tampoco se observan valores atípicos.

Pero al realizar el cruce con las variables x_1 y x_3 notamos el valor atípico.

- La distancia D_M de Mahalanobis: Para calcular la distancia de Mahalanobis de un conjunto de datos usando *Python*, usaremos la

**Figura 3.28**

```
import matplotlib.pyplot as plt

plt.plot(data.iloc[:,0], 'ro')
plt.ylabel('Rango')
plt.xlabel('Variable x1')
plt.show()
```

Figura 3.29

librería *Scipy* la cual trae por defecto una función para calcular la distancia de Mahalanobis. Primero debemos calcular la media de las columnas, usando la función `.mean()` de *Pandas*. A diferencia de *R*, debemos calcular la inversa de la matriz de covarianza (*Python* trae por defecto las filas y columnas intercambiadas con respecto a *R*), para ello usamos la funciones `.cov()`, la cual calcula la matriz de covarianza, `.linalg()` y `.inv()` para calcular la inversa. Luego, usando la función `apply()` de *Pandas* para aplicar la función `mahalanobis()` de la librería *Scipy* a todo el conjunto de datos. En la función `.apply()` debemos colocar el

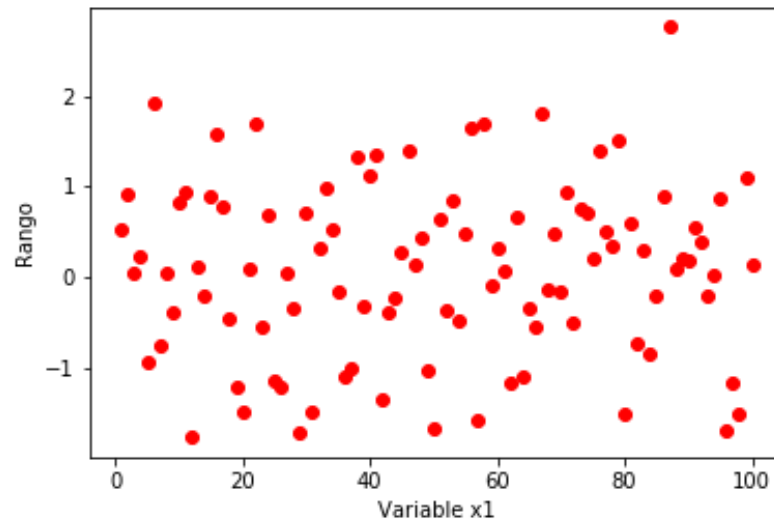


Figura 3.30

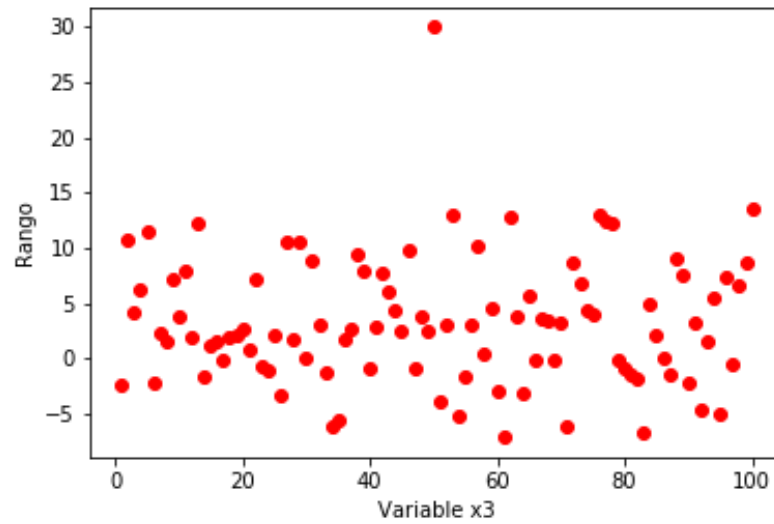


Figura 3.31

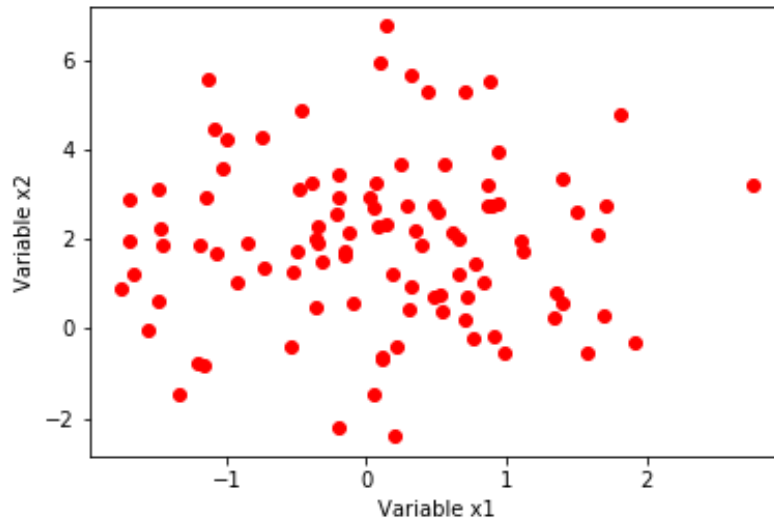


Figura 3.32

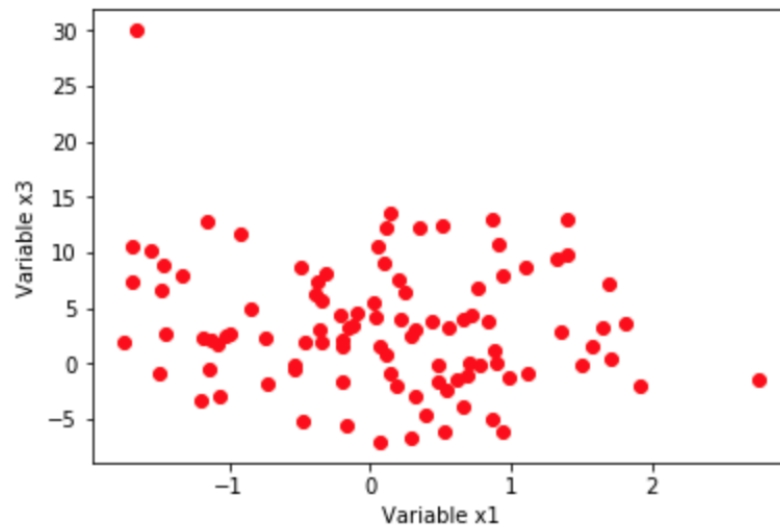


Figura 3.33

nombre de la función, *axis* para indicar si la función aplica a las columnas o filas: el valor de 0 para las columnas y 1 para las filas. Finalizamos con *args* que contiene los parámetros que necesita la función, en nuestro caso el punto de referencia, el cual es la media por columnas y la inversa de la matriz de covarianza.

```
import scipy.spatial.distance as distance

media = data.mean()

cov = numpy.cov(data.T)
Icov = numpy.linalg.inv(cov)

Mahalanobis = data.apply(distance.mahalanobis, axis=1,args=(media,Icov,))

plt.plot(Mahalanobis,'ro')
plt.ylabel('Rango')
plt.xlabel('Mahalanobis')
plt.show()
```

Figura 3.34

Podremos notar claramente un valor atípico en la siguiente imagen.

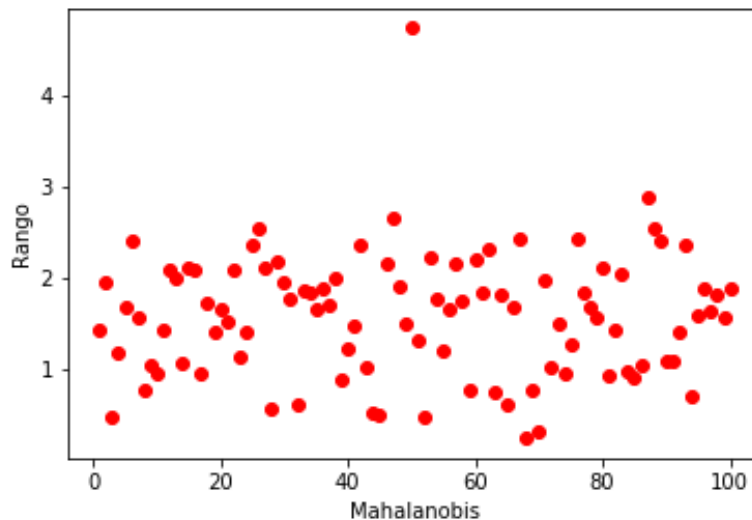


Figura 3.35

4

Regresión Lineal

La regresión lineal es un método estadístico que intenta predecir una variable de interés (dependiente) a partir de otras variables que se consideran influyentes (independientes) y se relacionan directamente con ella. Por ejemplo, hay estudios que sugieren la relación entre el tamaño del pétalo (P) y el sépalo de una flor (S), en algunas flores se aproxima el tamaño del pétalo multiplicando por 3 el tamaño del sépalo, es decir, $P = 3 * S$, esta relación es un ejemplo simple de lo que es la regresión lineal de solo dos variables.

En general si tenemos una variable y dependiente y n variables independientes $x_1, x_2, \dots, x_n$ un modelo de regresión lineal tiene la forma

$$y = c + c_1x_1 + c_2x_2 + \dots + c_nx_n$$

donde los coeficientes c_j son los coeficientes del modelo y c es el coeficiente independiente, en algunos casos es llamado intercept.

4.1. Identificando relación entre variables

Una de las primeras cosas a verificar antes de realizar el análisis de regresión es comprobar la relación existente entre la variable dependiente con las variables independientes, para lo cual existen diversas maneras de hacerlo:

- Visual: Se realiza un gráfico entre la variable dependiente y la variable independiente a estudiar, si se nota un tipo de linealidad entre los datos podemos pensar en incluir la variable indepen-

diente en el modelo. Por ejemplo, en el siguiente ejemplo se nota una cierta linealidad entre las variables.

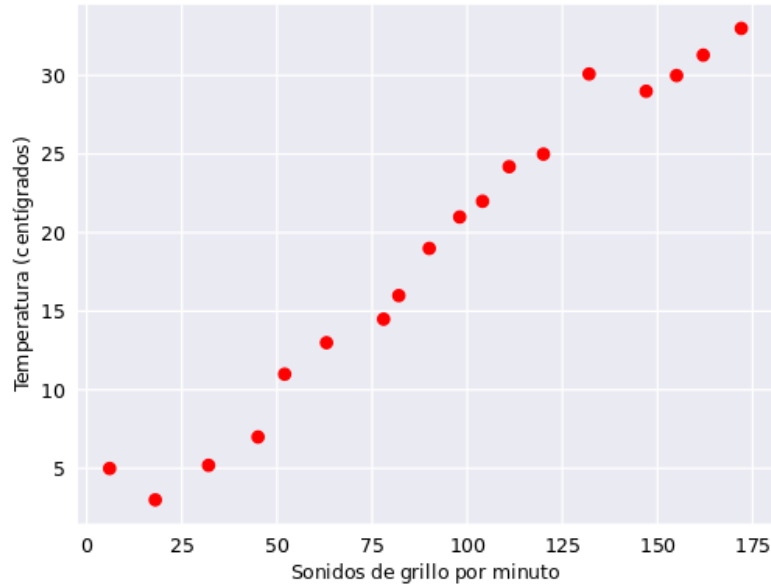


Figura 4.1

- Cuantitativa: Una manera cuantitativa de medir la relación entre variables es usar el índice de correlación, el cual varía entre -1 y 1. Entre más cerca de -1 o 1 esté el índice de correlación entre variables, mayor es su relación, en general, decimos que existe correlación lineal entre las variables independientes si el índice de correlación entre las variables es mayor a 0.5 o menor a -0.5.

Usaremos el conjunto de datos *mtcars* en *R* para ilustrar de ahora en adelante el proceso de regresión lineal. Los datos fueron extraídos de la revista Motor Trend US de 1974, y comprenden el consumo de combustible y 10 aspectos del diseño y rendimiento de los automóviles para 32 automóviles (modelos 1973-74).

4.1.1. Identificando relación entre variables con *R*

- Visual: Usamos la función *plot()* vista previamente para graficar el cruce entre variables, por ejemplo, entre las variables *mpg* (Millas/(US) por galón) y *disp* (Desplazamiento) notamos un cierto comportamiento lineal.

```
# Cargamos la base de datos  
data(mtcars)  
  
plot(mtcars$mpg,mtcars$disp, xlab = "mpg", ylab = "disp")
```

Figura 4.2

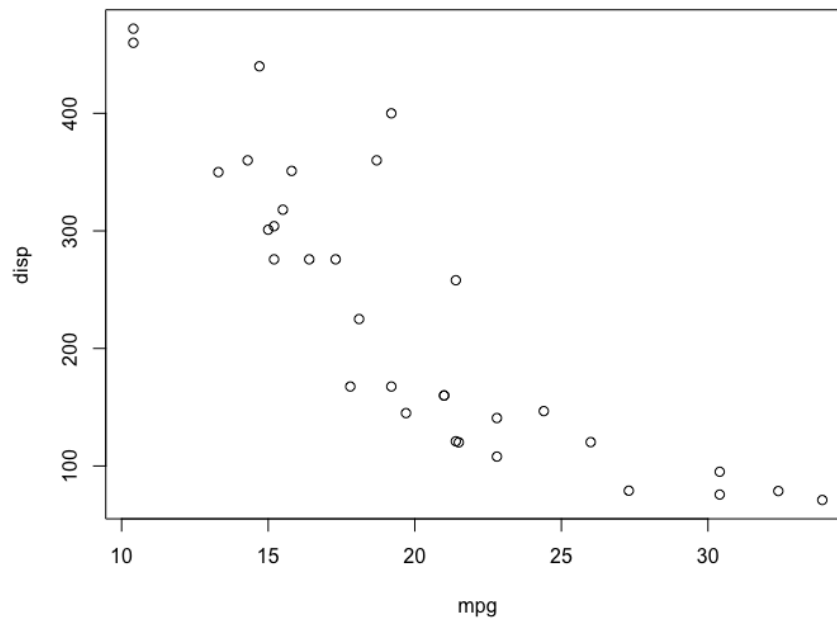


Figura 4.3

Con la función *plot()* tenemos la ventaja de que simplemente

pasando como argumento el conjunto de datos arroja una imagen con todos los cruces posibles entre las once (11) variables de la base de datos mtcars.

```
plot(mtcars)
```

Figura 4.4

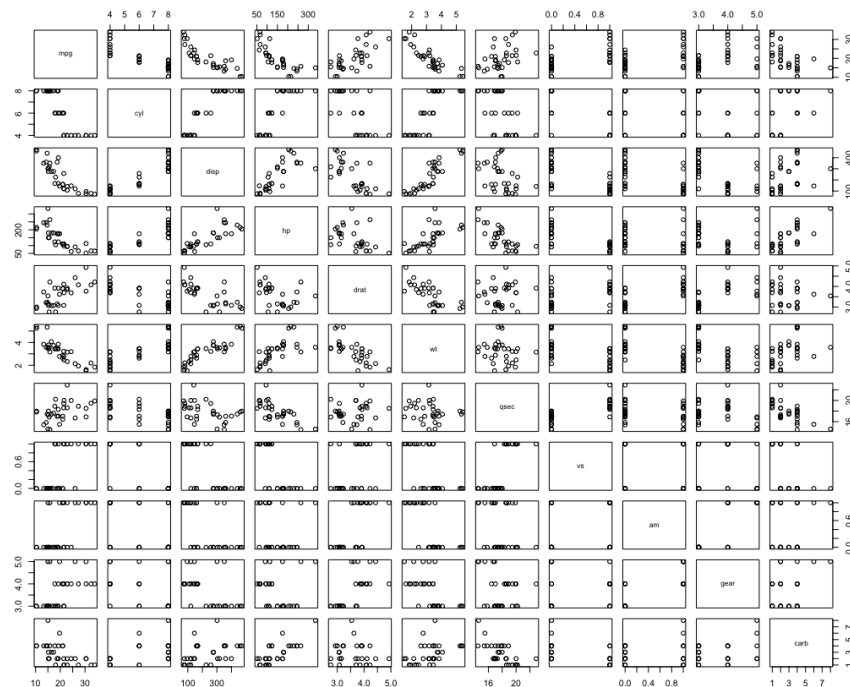


Figura 4.5

- Cuantitativa: La función `cor()` permite hallar la correlación entre las variables, por ejemplo entre `mpg` y `disp`

```
> cor(mtcars$mpg,mtcars$disp)
[1] -0.8475514
```

Este valor -0.84 nos confirma la posible relación que existe entre las variables `mpg` y `disp`. De igual manera podemos aplicar la función `cor()` a todo el conjunto de datos arrojando una matriz con todos los valores posibles de correlaciones.

```
> cor(mtcars)
```

	mpg	cyl	displacement	horsepower	drat	weight	qsec	vs	am	gear	carb
mpg	1.0000000	-0.8521620	-0.8475514	-0.7761684	0.6811719	-0.8676594	0.4186840	0.6640389	0.5998324	0.4802848	-0.5509250
cyl	-0.8521620	1.0000000	0.9020329	0.8324475	-0.6999381	0.7824958	-0.5912420	-0.8108118	-0.5226070	-0.4926866	0.5269882
displacement	-0.8475514	0.9020329	1.0000000	0.7909486	-0.7102139	0.8879799	-0.4336978	-0.7104159	-0.5912270	-0.5555692	0.3949768
horsepower	-0.7761684	0.8324475	0.7909486	1.0000000	-0.4487591	0.6587479	-0.7082239	-0.7230967	-0.2432042	-0.1257043	0.7498124
drat	0.6811719	-0.6999381	-0.7102139	-0.4487591	1.0000000	-0.7124406	0.0912047	0.4402785	0.7127113	0.6996101	-0.0907898
weight	-0.8676594	0.7824958	0.8879799	0.6587479	-0.7124406	1.0000000	-0.1747159	-0.5549157	-0.6924952	-0.5832870	0.4276059
qsec	0.4186840	-0.5912420	-0.4336978	-0.7082239	0.0912047	-0.1747159	1.0000000	0.7445354	-0.2298606	-0.2126822	0.6562492
vs	0.6640389	-0.8108118	-0.7104159	-0.7230967	0.4402785	-0.5549157	0.7445354	1.0000000	0.1683451	0.2060233	-0.5696071
am	0.5998324	-0.5226070	-0.5912270	-0.2432042	0.7127113	-0.6924952	-0.2298606	0.1683451	1.0000000	0.7940588	0.0575343
gear	0.4802848	-0.4926866	-0.5555692	-0.1257043	0.6996101	-0.5832870	-0.2126822	0.2060233	0.7940588	1.0000000	0.2740728
carb	-0.5509250	0.5269882	0.3949768	0.7498124	-0.0907898	0.4276059	0.6562492	-0.5696071	0.0575343	0.2740728	1.0000000

Figura 4.6

4.1.2. Identificando relación entre variables con *Python*

- Visual: En Python usaremos de nuevo la librería *Matplotlib*, para obtener el gráfico del cruce entre variables, de igual manera notaremos el comportamiento lineal entre las variables.

```
df = pandas.read_csv("mtcars.csv")
plt.plot(df.iloc[:,1],df.iloc[:,3], "ro")
```

Figura 4.7

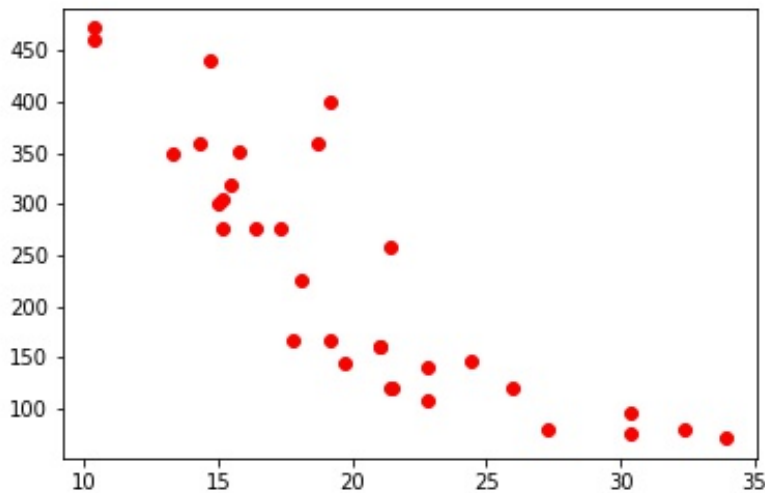


Figura 4.8

Ahora, si queremos realizar un solo gráfico con el cruce de todas las variables, como el caso de la función `cor()` en *R*, debemos implementar una librería llamada *Seaborn*, la cual arrojará el resultado deseado haciendo uso de la función `.pairplot()`, con los argumentos: `kind` (para el tipo de gráfico) y `palette` (para el color).

```
import seaborn as sns
sns.pairplot(df, kind="scatter", palette="Set2")
plt.show()
```

Figura 4.9

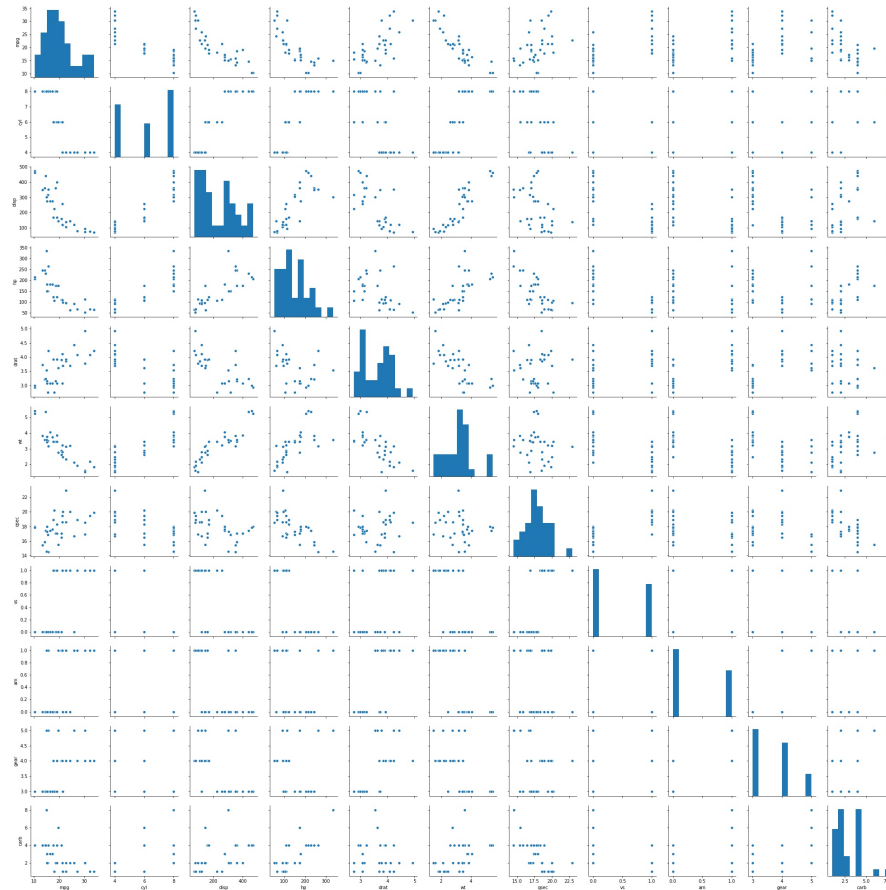


Figura 4.10

- **Cuantitativo:** Para obtener la matriz de correlación entre las variables de nuestro conjunto de datos, hacemos uso de la función `.corr()` de la librería *Pandas*. En el caso que queramos alguna correlación en específico filtramos los elementos de la matriz anterior.

```
corre = df.corr()
corre
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear
mpg	1.000000	-0.852162	-0.847551	-0.776168	0.681172	-0.867659	0.418684	0.664039	0.599832	0.480285
cyl	-0.852162	1.000000	0.902033	0.832447	-0.699938	0.782496	-0.591242	-0.810812	-0.522607	-0.492687
disp	-0.847551	0.902033	1.000000	0.790949	-0.710214	0.887980	-0.433698	-0.710416	-0.591227	-0.555569
hp	-0.776168	0.832447	0.790949	1.000000	-0.448759	0.658748	-0.708223	-0.723097	-0.243204	-0.125704
drat	0.681172	-0.699938	-0.710214	-0.448759	1.000000	-0.712441	0.091205	0.440278	0.712711	0.699610
wt	-0.867659	0.782496	0.887980	0.658748	-0.712441	1.000000	-0.174716	-0.554916	-0.692495	-0.583287
qsec	0.418684	-0.591242	-0.433698	-0.708223	0.091205	-0.174716	1.000000	0.744535	-0.229861	-0.212682
vs	0.664039	-0.810812	-0.710416	-0.723097	0.440278	-0.554916	0.744535	1.000000	0.168345	0.206023
am	0.599832	-0.522607	-0.591227	-0.243204	0.712711	-0.692495	-0.229861	0.168345	1.000000	0.794059
gear	0.480285	-0.492687	-0.555569	-0.125704	0.699610	-0.583287	-0.212682	0.206023	0.794059	1.000000
carb	-0.550925	0.526988	0.394977	0.749812	-0.090790	0.427606	-0.656249	-0.569607	0.057534	0.274073

Figura 4.11

4.2. Transformando variables

Una de las principales hipótesis en la regresión lineal es la normalidad de las variables implicadas, es decir, que las variables provengan de una distribución normal. Pero, ¿cómo identificamos este comportamiento en una variable? La forma más sencilla es inspeccionando el histograma proveniente de dicha variable y corroborando si existe un comportamiento acampanado, como en el siguiente ejemplo:

Pero en muchos casos, las variables no poseen este comportamiento, por lo tanto se debe aplicar transformaciones que imiten el comportamiento acampanado de la gráfica anterior. Las transformaciones más comunes son: $y = x^2$, $y = \sqrt{x}$, $y = \ln(x)$ y $y = \frac{1}{x}$. En las

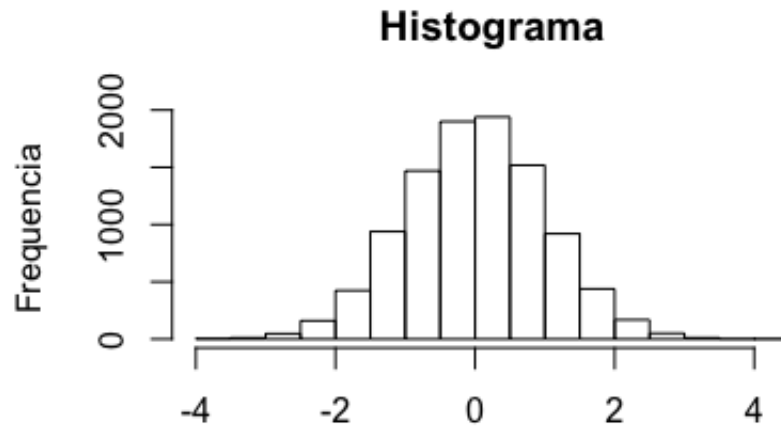


Figura 4.12

siguientes gráficas podemos apreciar el efecto de estas transformaciones:

Esta última imagen nos da una idea de cuando debemos aplicar alguna de las transformaciones anteriores. Ahora veremos cómo aplicarlas en los respectivos lenguajes

4.2.1. Aplicando transformaciones de variables en *R*

Esta tarea es muy fácil de ejecutar en *R*, supongamos que estamos trabajando con la variable x del conjunto de datos df , los códigos para realizar estas transformaciones son:

4.2.2. Aplicando transformaciones de variables en *Python*

En Python es fácil realizar las transformaciones menos la del logaritmo, para la cual usaremos la librería *Math* en específico la

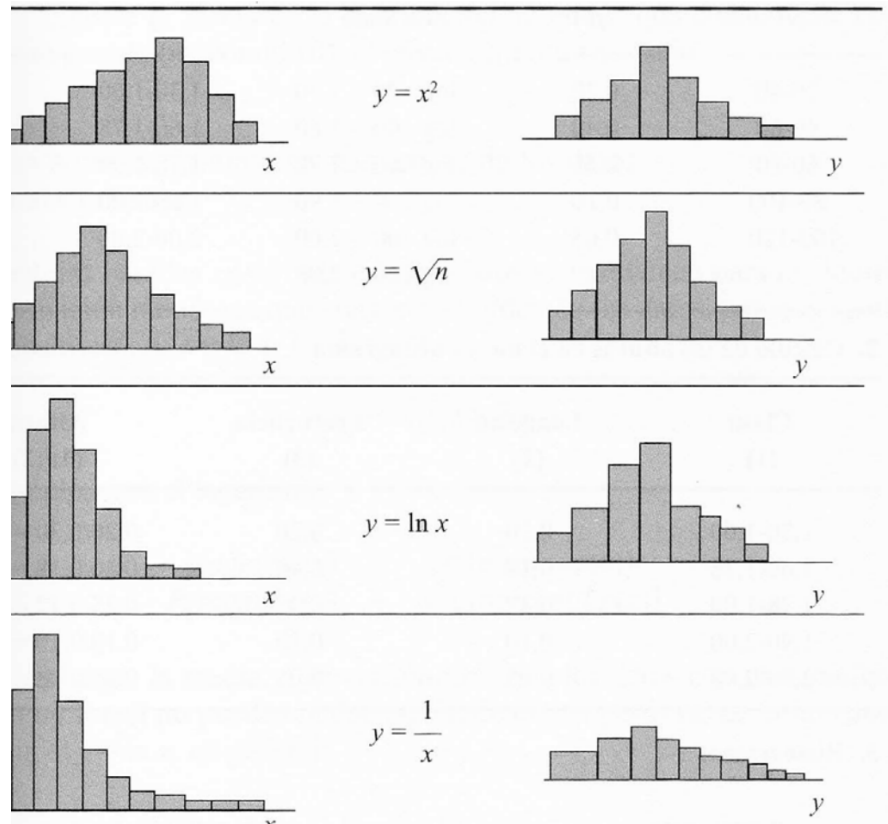


Figura 4.13

```

### Transformación  $x^2$ 
df$x=df$x^2
## Visualizamos el cambio
head(df$x)

### Transformación raíz de  $x$ 
df$x=sqrt(df$x)
## Visualizamos el cambio
head(df$x)

### Transformación  $1/x$ 
df$x=1/df$x
## Visualizamos el cambio
head(df$x)

### Transformación  $\lg(x)$ 
df$x=log(df$x)
## Visualizamos el cambio
head(df$x)

```

Figura 4.14

función `.log()` junto a la función `.apply`, pues la función `.log()` se aplica a números y queremos aplicarla a una variable completa.

```
### Función raíz de X
df[["x"]] = df[["x"]]**(1/2)

### Función 1/X
df[["x"]] = 1/df[["x"]]

### Función log(x)
import math
df[["x"]] = df[["x"]].apply(math.log, axis=1)
```

Figura 4.15

4.3. Eliminando el efecto de colinealidad entre variables independientes.

Otra hipótesis importante antes de realizar el modelo de regresión, es la no colinealidad entre las variables, es decir, inexistencia de una relación lineal entre variables independientes. En caso de descubrir este efecto debemos eliminar una de estas variables y quedarnos con la otra.

La forma de identificar colinealidad es exactamente igual a la forma de encontrar correlaciones entre las variables vistas al principio de este capítulo. Primero calculamos el índice de correlación entre variables, si se encuentra una alta correlación entre un par de variables independientes se procede a revisar la gráfica del cruce de dichas variable, y si se observa una clara relación lineal procedemos a eliminar alguna de ellas.

4.4. Eliminación de las variables categóricas

En los modelos de regresión lineal las variables categóricas no cumplen las hipótesis del modelo, pues no satisfacen el supuesto de normalidad. Recordemos que una variable categórica es una variable que toma unos valores fijos ya conocidos, por ejemplo, la variable que toma los valores 1,2,3,4,5,6 que representa los posibles valores que toma el lanzamiento de un dado, es una variable categórica, o la variable que toma los valores de Si o No que se refiere al consumo de un producto específico. Por lo tanto debemos saber cómo eliminar variables de nuestro conjunto de datos.

4.4.1. Eliminación de variables en *R*

Supongamos que tenemos en un conjunto de datos llamados *mtcars* la variable *mpg* la cual deseamos eliminar. Para ello usamos el siguiente código, en el cual el símbolo *!* es el indicador para eliminar dicha variable.

```
mtcars<-mtcars[ ,!colnames(mtcars)== "mpg"]
```

Figura 4.16

Para recuperar la columna eliminada se vuelve a cargar la base de datos *data(mtcars)*.

4.4.2. Eliminación de variables en *Python*

De igual manera suponiendo que tenemos el mismo conjunto de datos y la misma variable a eliminar en *Python*, usamos la función

`.drop()` para eliminar la variable, y denotamos `axis=1` para indicar que estamos eliminando una columna

```
mtcars = pandas.read_csv("mtcars.csv")
mtcars = mtcars.drop("mpg", axis=1)
mtcars.head()
```

	Unnamed: 0	cyl	dis	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	8	360.0	175	3.15	3.440	17.02	0	0	3	2

Figura 4.17

4.5. Aplicando el modelo de regresión lineal

Una vez que elegimos las variables que deseamos usar en el modelo debemos conocer que funciones y parámetros utilizar.

4.5.1. Aplicando el modelo de regresión lineal en *R*

En *R* existen varias formas de aplicar este modelo, gracias al gran número de librerías existentes, sin embargo, ya *R* trae por defecto la función `lm()` la cual nos permite construir el modelo de una forma sencilla.

Primero describiremos los parámetros más importantes que debemos ingresar:

- *formula*: Sin duda el parámetro más importante. Es el que le indica a la función cuales son las variables independientes y la variable dependiente, por ejemplo si tenemos las variables x_1 , x_2 , x_3 donde las variables independientes son x_2 y x_3 y la variable dependiente es x_1 , el parámetro formula debe ser : $x_1 \sim x_2 + x_3$.

En el caso de que deseamos usar todas las variables del conjunto de datos como independientes, simplemente usamos $x_1 \sim$.

- *data*: El conjunto de datos a utilizar para realizar el modelo.
- *subset*: Un vector donde se especifican las filas a usar del modelo.
- *weights*: En el ajuste del modelo se calculan los parámetros que representan a cada variable independiente. Para realizar dicho ajuste se realiza un método de optimización para obtener los parámetros que mejor se ajusten a la recta de regresión. El parámetro *weights* es un vector numérico que se encarga de mejorar esta aproximación y debe coincidir con el número de observaciones
- *na.action*: En el caso de que existan valores *NAs*, este parámetro acepta una función que indica que hacer con estos valores problemas, lo usual es utilizar *na.fail*, *na.omit* o *na.exclude*
- *method*: Hace referencia al método de aproximación de los parámetros de los coeficientes. El método por defecto es el de los mínimos cuadrados, el cual se denota por *qr*

Ahora supongamos que tenemos un conjunto de datos llamados *irisP* el cual cuenta con la variable dependiente *Sepal.Length* y las variables independientes *Sepal.Width*, *Petal.Length*, *Petal.Width*. Para realizar el modelo con los parámetros ya mencionados tenemos dos opciones, la primera colocando el nombre de todas las variables o haciendo uso de la abreviación de la formula ya mencionado.

```
lm(Sepal.Length ~ Sepal.Width + Petal.Length + Petal.Width, data = irisP, weights = rep(1:150), na.action = na.omit, method = "qr")
# o también
lm(Sepal.Length ~ ., data = irisP, weights = rep(1:150), na.action = na.omit, method = "qr")
```

Figura 4.18

Si hacemos uso del código anterior obtendremos la información básica del modelo, la cual se presenta en la siguiente imagen:

En esta imagen se nos proporciona el valor que se introdujo en el modelo y los coeficientes de la regresión. Si queremos los datos del modelo usamos la función *summary()*

```
Call:
lm(formula = Sepal.Length ~ ., data = irisP, weights = rep(1,
  150), na.action = na.omit, method = "qr")

Coefficients:
(Intercept)   Sepal.Width   Petal.Length   Petal.Width
      1.8560      0.6508      0.7091     -0.5565
```

Figura 4.19

```
> modelo <- lm(Sepal.Length~ . ,data = irisP, weights = rep(1,150),na.action = na.omit,method = "qr")
> summary(modelo)

Call:
lm(formula = Sepal.Length ~ ., data = irisP, weights = rep(1,
  150), na.action = na.omit, method = "qr")

Residuals:
    Min       1Q   Median       3Q      Max
-0.82816 -0.21989  0.01875  0.19709  0.84570

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.85600    0.25078   7.401 9.85e-12 ***
Sepal.Width  0.65084    0.06665   9.765 < 2e-16 ***
Petal.Length 0.70913    0.05672  12.502 < 2e-16 ***
Petal.Width -0.55648    0.12755  -4.363 2.41e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3145 on 146 degrees of freedom
Multiple R-squared:  0.8586,    Adjusted R-squared:  0.8557
F-statistic: 295.5 on 3 and 146 DF,  p-value: < 2.2e-16
```

Figura 4.20

4.5.2. Aplicando el modelo de regresión lineal en *Python*:

Existen varias librerías para aplicar modelos lineales en *Python*, entre las más populares están *StatsModels* y *sklearn*. Usaremos la primera por ser la que nos proporciona unas funciones parecidas a las que tenemos en *R*. Como el objetivo de este curso es aprender a usar la herramienta, no incluiremos algunos parámetros como los que describimos en *R*.

Para crear el modelo usando la librería *StatsModels*, debemos importar la librería de la manera usual. Luego seleccionamos la variable dependiente e independientes, a diferencia de *R*, *StatsModel* no calcula el valor del intercept por defecto, por lo tanto debemos

especificarlo de manera explícita. Esto lo hacemos con la función `add_constant()` de la misma librería, luego usando la función `.OLS()` para crear el modelo y la función `.fit()` para entrenarlo. Finalmente imprimimos los datos del modelo usando la función `.summary()`

```
Subimos la base de datos "iris"

iris = pandas.read_csv("iris.csv",sep=";")

Importamos la librería "statsmodels"

import statsmodels.api as sm

Seleccionamos las variables dependiente e independientes

var_dep = iris["Sepal.Length"]
var_ind = iris.drop("Sepal.Length",axis=1)

Agregamos el cálculo del intercept

var_ind1 = sm.add_constant(var_ind)
```

Figura 4.21

Creamos y entrenamos el modelo

```
model = sm.OLS(var_dep, var_ind1).fit()
print(model.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          Sepal.Length    R-squared:                0.859
Model:                OLS              Adj. R-squared:           0.856
Method:             Least Squares      F-statistic:             295.5
Date:                Sat, 18 May 2019   Prob (F-statistic):      8.59e-62
Time:                  14:11:47         Log-Likelihood:         -37.321
No. Observations:        150           AIC:                   82.64
Df Residuals:           146           BIC:                   94.69
Df Model:                 3
Covariance Type:        nonrobust
=====
                    coef    std err          t      P>|t|      [0.025    0.975]
-----
const                1.8560      0.251      7.401    0.000      1.360      2.352
Sepal.Width          0.6508      0.067      9.765    0.000      0.519      0.783
Petal.Length         0.7091      0.057     12.502    0.000      0.597      0.821
Petal.Width         -0.5565      0.128     -4.363    0.000     -0.809     -0.304
=====
Omnibus:                 0.345    Durbin-Watson:           2.060
Prob(Omnibus):           0.842    Jarque-Bera (JB):         0.504
Skew:                    0.007    Prob(JB):                 0.777
Kurtosis:                2.716    Cond. No.                  54.7
=====
```

Figura 4.22

4.6. Cómo interpretar los resultados del modelo

Ya vimos que una vez construido el modelo usando funciones con el nombre de *summary* obtenemos información sobre características de los modelos. Describiremos las más importantes y como entender su efecto en el modelo.

4.6.1. Interpretar con *summary* en *R*:

- *residuals*: Al calcular el modelo, la función *summary* calcula las predicciones para los mismos datos con los que se entrenó el modelo y la variable *residuals* nos da los errores de predicción. Con esta variable se pueden obtener el menor, el mayor, la mediana, primer cuantil y tercer cuantil de los errores. Debemos realizar un histograma de los errores del modelo, si este histograma es distinto a un comportamiento normal, debemos pensar rápidamente que nuestro modelo es inadecuado. A continuación se muestra el código y un histograma ideal, suponiendo que usamos el “modelo” obtenido en la sección anterior

```
hist(modelo$residuals,main = "Histograma de errores",xlab = "",ylab="")
```

Figura 4.23

- *Coefficients*: Luego se muestran los coeficientes del modelo en forma de matriz:
 - En la primera columna se muestran los coeficientes del modelo.
 - En la segunda columna se muestran las desviaciones estándares de cada coeficiente.
 - Luego se muestran los valores correspondientes al estadístico *t*, con el fin de realizar una prueba de hipótesis a los coeficientes.

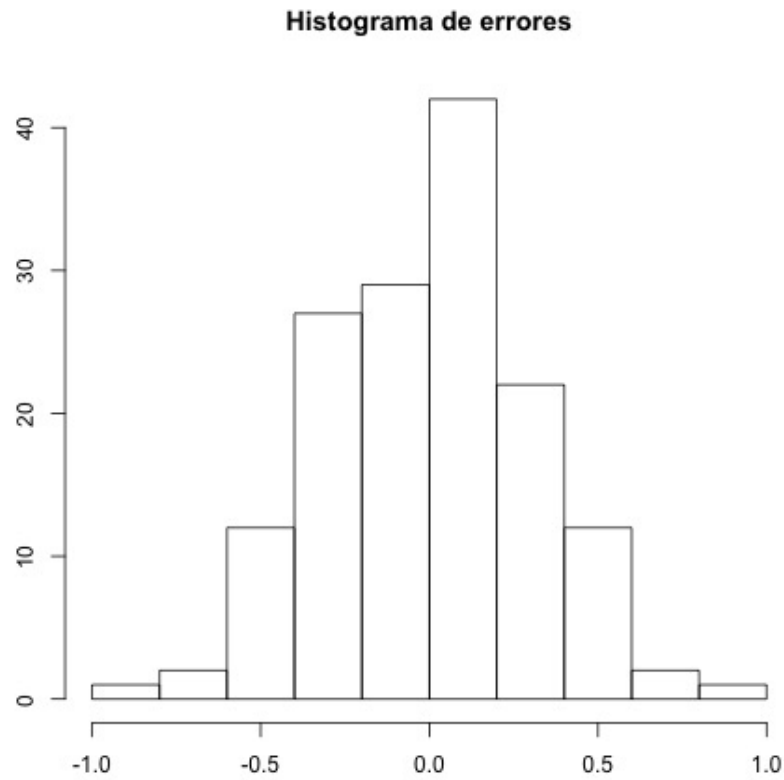


Figura 4.24

- En cuarta columna se muestran los p -valores de la prueba de hipótesis, los resultados de estos valores deben ser lo más pequeños posibles, en general, se esperan que sean menores a 0.05, en caso de que esto no ocurra, debemos pensar en eliminar la variable del modelo, o realizar alguna transformación adecuada.
- *Residual standard error*: Nos da una idea de cómo varía el error de precisión del modelo, es decir, nos dice como varían las predicciones de las observaciones.
- *Multiple R-squared* and *Adjusted R-squared* : En ambos casos

ambos valores explican el valor de la varianza explicada del modelo. Se diferencian en que el segundo parámetro considera el número de variables involucradas en el modelo, por lo cual en la mayoría de los casos se utiliza con preferencia sobre el primero y se buscan valores mayores a 0.7 o 0.8.

- *F-statistic*: Nos presenta el valor del estadístico F , luego los grados de libertad y por último el p -valor de la prueba de hipótesis que resulta de contrastar el modelo resultante con el modelo que únicamente consta de un término intercept

4.6.2. Interpretar con *summary* en *Python*

El *summary* en este caso nos da mucha más información que en *R*. Nos concentraremos en las mismas variables del *summary* y mencionaremos algunas otras que pueden ser de utilidad. Este *summary* se divide en dos cuadros:

- Información estadística del modelo: en esta primera matriz la información más relevante es el nombre de la variable dependiente, el número de observaciones, *Multiple R-squared*, *Adjusted R-squared* y el valor de *F-statistic*
- Información estadística acerca de los coeficientes del modelo: en esta matriz se encuentra prácticamente la misma información concerniente a los valores de los coeficientes del modelo, con la información extra que consiste en los intervalos de confianza de los coeficientes.
- Este *summary* no aporta información clara sobre los residuos del modelo. Para poder obtener un histograma y ver su comportamiento usamos el siguiente código, recordando que nuestro modelo se llama “modelo”:

```
plt.hist(model.resid)
```

Figura 4.25

4.7. Realizando predicciones con nuestro modelo

El objetivo principal de crear modelos es usarlos para predecir valores a partir de las variables independientes. A continuación presentaremos los códigos para realizar dicha predicción:

4.7.1. Realizando predicciones con *R*

Para realizar predicciones con *R* haremos uso de la función *predict()*. Debemos ser cuidadosos con los conjuntos de datos que usaremos para predecir, estos datos deben tener las columnas con el mismo nombre y no tener valores faltantes o espacios en blanco. A continuación se anexa el código para hacer la predicción.

```
predicciones<-predict(modelo, irisP)
predicciones
```

Figura 4.26

4.7.2. Realizando predicciones con *Python*

Al realizar predicciones en *Python* debemos tener en claro el orden del conjunto de datos, es decir, debemos tener las variables independientes en el mismo orden con el que se entrenó el modelo, además, debemos agregar una variable intercept para que el modelo tenga todos los parámetros requeridos.

Agregando variable constante para el intercept

```
pre = sm.add_constant(pre)
```

```
/Users/arturocarreno/anaconda3/lib/python3.6/site-packages/numpy/core/fromnumeric.py:2389: Future  
Warning: Method .ptp is deprecated and will be removed in a future version. Use numpy.ptp instead  
.  
    return ptp(axis=axis, out=out, **kwargs)
```

Predicciones

```
predicciones = model.predict(pre)
```

Figura 4.27

5

Regresión logística

La regresión logística nos da respuesta a la siguiente interrogante, ¿Qué hacer cuando la variable dependiente es dicotómica? Claramente en los modelos lineales esto es inaplicable pues la variable dependiente debe ser continua. De igual manera la otra interrogante importante es ¿Qué hacer cuando las variables independientes son categóricas? La regresión logística da la flexibilidad necesaria para incluir este tipo de variables. La regresión logística calcula la probabilidad de que el evento ocurra, y en esto se diferencia de la regresión lineal en la cual se calculaba directamente el valor de la variable. Por ejemplo, si estamos interesados en el evento cara o cruz, un modelo logístico para este tipo de modelos nos dará la probabilidad de que ocurra cara o cruz.

5.1. Supuestos de la regresión logística

La regresión logística es flexible con respecto a supuestos como normalidad o la continuidad de las variables. Una de las hipótesis que debemos verificar es la inexistencia de multicolinealidad entre las distintas variables continuas y la correlación entre las variables independientes y la variable dependiente. A continuación veremos cómo responder a estas interrogantes.

5.2. Multicolinealidad entre variables independientes

Al igual que en la regresión lineal debemos evitar variables independientes con alta correlación. Para ello debemos realizar el mismo procedimiento visto en el capítulo anterior.

5.3. Detectar correlación entre variables categóricas

Una de las primeras cosas que debemos revisar es la independencia entre las variables independientes categóricas y la dependencia entre la variable dependiente y las variables categóricas independientes. Para ello haremos uso de las tablas de contingencia.

Una tabla de contingencia es la forma más fácil de detectar relaciones entre variables categóricas, en estas tablas se definen las variables en filas y columnas y se ven las distintas proporciones entre las categorías de las variables. En la siguiente imagen se muestra un ejemplo de tabla de contingencia

FACTORES	Usa Internet	No usa Internet	TOTAL
Hombres	84	36	120
Mujeres	56	24	80
TOTAL	140	60	200

Figura 5.1

En las columnas la variable hace referencia al uso del internet y en las filas las variables hacen referencia al sexo.

Para detectar si dos variables están o no correlacionadas usaremos

la prueba *chi-cuadrado* que contrasta la independencia entre las variables, es decir, si el *p-valor* es bajo estamos encontrando evidencia de independencia entre los factores, en caso contrario podemos suponer la existencia de cierta relación entre las variables.

En la siguiente imagen se muestra una tabla de contingencia, a la cual le aplicaremos la prueba *chi-cuadrado*, para contrastar la independencia entre el uso de internet y el estar empleado. La tabla se construye con el siguiente código:

```
d<-matrix(c(20,36,30,32),nrow=2,byrow=T)
dimnames(d)<-list(c("Trabaja","No Trabaja"), c("Usa Internet","No Usa Internet"))
```

Figura 5.2

```
> d
```

	Usa Internet	No usa internet
Trabaja	20	36
No trabaja	30	32

Figura 5.3

5.3.1. Prueba chi-cuadrado en R

Para realizar la prueba usamos la función *chisq.test()* y obtenemos el siguiente resultado

```
> chisq.test(d)
```

Pearson's Chi-squared test with Yates' continuity correction

data: d
X-squared = 1.451, df = 1, p-value = 0.2284

Figura 5.4

El valor que interesa es el *p-valor*, el cual al ser mayor a 0.05 indica

una posible relación entre las variables. En general, si el *p-valor* es cercano a uno podemos pensar en una posible relación entre las variables.

5.3.2. Prueba chi-cuadrado en *Python*

Para realizar la prueba *chi-cuadrado* en Python usaremos la librería *scipy* y usaremos la función *.chi2_contingency()*. A continuación mostramos como hacerlo

Cargamos la librería

```
import scipy.stats as stats
```

Realizamos e imprimimos el valor del *p-valor*

```
d = pandas.read_csv("prueba.txt", index_col=0)
chi2, p, dof, expected = stats.chi2_contingency(d)
print(p)
```

```
0.2283643947212718
```

Figura 5.5

5.4. Detectar correlación entre variables categóricas y variables cuantitativas

Para detectar un posible efecto entre la correlación de una variable categórica y una variable continua usaremos un estadístico llamado *d de Cohen*, el cual se construye a partir de las diferencias de las medias de la segmentación a través de la variable categórica. Dependiendo del valor del estadístico diremos si existe o no un efecto de la variable categórica y la variable cuantitativa. En general si el valor es menor 0.5 diremos que hay un efecto débil, si el valor está entre 0.5 y 0.8 diremos que hay un efecto moderado y si es mayor a 0.8 diremos que hay un efecto fuerte. En general este estadístico es útil cuando la variable categórica posee dos categorías, en otro

caso, el estadístico pierde utilidad y se suele usar una herramienta estadística conocida como *Anova*, la cual veremos en otro curso.

5.4.1. Medida de d de Cohen en *R*

Deberemos instalar la librería *effsize* y usaremos la función *cohen.d()*, además crearemos una variable artificial para mostrar el uso del estadístico:

```
library(effsize)

f<-c(rep(1,50),rep(0,25),rep(2,25))
g<-c(rnorm(50,43,23),rnorm(50))
cohen.d(g,f)
```

Figura 5.6

```
Cohen's d

d estimate: 1.073829 (large)
95 percent confidence interval:
  lower      upper 
0.775521 1.372137
```

Figura 5.7

Podemos notar que el valor del estadístico d de Cohen es alto mostrando un posible efecto de las variables categóricas en la variable continua.

5.4.2. Medida de d de Cohen en *Python*

En *Python* no existe una función para calcular la d de Cohen de manera directa por lo que tendremos que programarla por nuestra cuenta, para ello usaremos las librerías *statistics* y *math* las cuales nos permitirán calcular las métricas que necesita el estadístico de Cohen.

```
from statistics import mean, stdev
from math import sqrt

c0 = [2, 4, 7, 3, 7, 35, 8, 9]
c1 = [4, 8, 14, 6, 14, 70, 16, 18]

cohens_d = (mean(c0) - mean(c1)) / (sqrt((stdev(c0) ** 2 + stdev(c1) ** 2) / 2))

print(cohens_d)
```

Figura 5.8

5.5. Codificando variables categóricas

Una variable categórica no puede ser ingresada directamente al modelo, pues se asume que es una variable continua, por lo cual arrojará una interpretación errónea del comportamiento de la variable. El procedimiento estándar es crear variables auxiliares binarias que nos indiquen la presencia o no de cierto atributo. Por ejemplo, si tenemos una variable categórica con tres categorías: rojo, azul y verde, debemos crear dos variables auxiliares: la primera que esté compuesta por 0 si es roja, 1 si no, la segunda que esté compuesta por 1 si es azul 0 si no. Es innecesario crear una tercera variable para el color verde, pues se entiende que si las primeras variables son 0 y 0, se concluye que debe ser verde. En general si una variable categórica está compuesta por n categorías debemos crear $n - 1$ variables auxiliares, en algunas fuentes esto también se conoce como creación de variables *Dummies*

5.5.1. Creación de variables auxiliares en *R*

Debemos descargar y usar la librería *fastDummies*, la cual tiene la función del mismo nombre *dummy_cols()*. Esta función es la que crea las variables auxiliares. Por ejemplo, supongamos que tenemos la variable color, la cual está compuesta por: (“azul”, “rojo”, “amarillo”, “blanco”, “amarillo”, “azul”, “azul”, “rojo”), en la cual tenemos 4 categorías. Para aplicar la función *dummy_cols()* realizaremos el siguiente código:

```
library(fastDummies)

color<-c("azul","rojo","amarillo","blanco","amarillo", "azul","azul","rojo")

colorD<-dummy_cols(color, remove_first_dummy=TRUE)[-1]
```

Figura 5.9

Y obtenemos como resultado:

.data_rojo	.data_amarillo	.data_blanco
0	0	0
1	0	0
0	1	0
0	0	1
0	1	0
0	0	0
0	0	0
1	0	0

Figura 5.10

Observamos que en el séptimo registro tenemos una fila de números ceros, por lo ya señalado esto significa que el atributo de esta observación es azul.

5.5.2. Creación de variables auxiliares en *Python*

Para crear variables auxiliares en *Python*, utilizaremos función `get_dummies()` del paquete Pandas. A continuación mostramos el código y el resultado.

```
import pandas
color=["azul", "rojo", "amarillo","blanco","amarillo","azul","azul","rojo"]
dummies=pandas.get_dummies(color)
dummies = dummies.drop(dummies.columns[[0]],axis=1)
dummies
```

	azul	blanco	rojo
0	1	0	0
1	0	0	1
2	0	0	0
3	0	1	0
4	0	0	0

Figura 5.11

5.6. Aplicando el modelo de regresión logística

Una vez realizado un paseo por los pasos anteriores debemos proceder a realizar la construcción del modelo como tal. Iniciaremos con el lenguaje *R*:

5.6.1. Modelo logístico en R

Para el cálculo del modelo logístico haremos uso de la función *glm()* que está disponible como función base en R. Esta función tiene prácticamente la misma sintaxis que la función *lm()*, así que no describiremos en detalle sus parámetros.

Usaremos los datos *Default* (incumplimiento) que nos proporciona el paquete *ISLR* sobre registros de clientes con incumplimiento de pagos de una empresa de tarjetas de crédito, los cuales podemos observar en la siguiente imagen:

Si queremos crear un modelo logístico para predecir el incumplimiento de pago de los estudiantes que tenga variables independientes *balance*, *income* y variable dependiente *default* debemos usar el siguiente código

tenemos un parámetro adicional llamado *Family* el cual hace re-

```
library(ISLR)  
head(Default)
```

Figura 5.12

default	student	balance	income
No	No	729.52650	44361.625
No	Yes	817.18041	12106.135
No	No	1073.54916	31767.139
No	No	529.25060	35704.494
No	No	785.65588	38463.496
No	Yes	919.58853	7491.559
No	No	825.51333	24905.227
No	Yes	808.66750	17600.451

Figura 5.13

```
modelo_logistico <- glm(default~balance+student, data=Default, family="binomial")
```

Figura 5.14

ferencia a la distribución de la variable que en nuestro caso es la binomial.

Al realizar un resumen del modelo con la función *summary*, obtenemos:

```
> summary(modelo_logistico)

Call:
glm(formula = default ~ balance + student, family = "binomial",
    data = datos)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.4578  -0.1422  -0.0559  -0.0203   3.7435

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.075e+01  3.692e-01 -29.116  < 2e-16 ***
balance      5.738e-03  2.318e-04  24.750  < 2e-16 ***
studentYes   -7.149e-01  1.475e-01  -4.846  1.26e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2920.6  on 9999  degrees of freedom
Residual deviance: 1571.7  on 9997  degrees of freedom
AIC: 1577.7

Number of Fisher Scoring iterations: 8
```

Figura 5.15

La imagen anterior es una presentación del resumen del modelo un poco diferente a la del capítulo anterior. Vale la pena describir la variable *AIC*, este estadístico nos da información acerca de la bondad del modelo con respecto a las variables introducidas, es decir, permite comparar distintos modelos estadísticos, buscando un valor lo más pequeño posible de esta variable.

Un error típico en el que se suele caer al crear un modelo de regresión logística es pensar que este modelo dará como predicción alguna variable categórica específica. En realidad el modelo proporciona la probabilidad de ocurrencia del evento con los datos específicos. Así, la predicción esperada será un número entre 0 y 1.

A continuación presentamos el código para realizar la predicción:

```
> observacion = Default[9999,]  
> predict(modelo_logistico, observacion , type="response")  
9999  
0.1485071
```

Figura 5.16

En la imagen anterior el resultado es 0,14 significa que la probabilidad de ocurrencia del evento es 0,14.

5.6.2. Modelo logístico en *Python*

Para crear el modelo cargaremos la librería *pandas* y *statsmodels*. Usaremos los mismos datos de la parte anterior. A diferencia de *R* debemos separar la variable dependiente de las independientes y verificar que las variables categóricas estén compuestas por valores numéricos. Estos cambios son hechos con la función *.replace()*. Recordemos que por defecto no se calcula la variable intercept en el modelo, por lo cual debemos incorporarla con la función *sm.add_constant()*. A continuación se muestra este procedimiento:

Ahora, procedemos al cálculo del modelo. Primero lo creamos usando la función *sm.logit()* y luego lo entrenamos usando la función *.fit()*. Para ver el resumen del modelo usamos la función *.summary()*, la cual nos da prácticamente la misma información que la función *summary* en *R*.

Para concluir, realizamos predicciones usando nuestro modelo, recordando que la observación usada para predecir debe tener la misma estructura de la variable independiente. Debemos tener en cuenta que dependiendo de si se agregó o no el intercept, tendremos que agregar la variable constante usando la función *sm.add_constant()*. A continuación se muestra este procedimiento:

Cargamos las librerías y los datos

```
import pandas as pd
import statsmodels.api as sm

data = pd.read_csv("logistica.csv")
data.head()
```

	default	student	balance	income
0	No	No	729.526495	44361.625074
1	No	Yes	817.180407	12106.134700
2	No	No	1073.549164	31767.138947
3	No	No	529.250605	35704.493935
4	No	No	785.655883	38463.495879

Seleccionamos las variables independientes y dependiente

```
Y = data[["default"]]
X = data[["student", "balance"]]
```

Modificamos las variables categóricas

```
X = X.replace("No", 0).replace("Yes", 1)
X = sm.add_constant(X)
Y = Y.replace("No", 0).replace("Yes", 1)
```

Figura 5.17

Creamos y ajustamos el modelo

```
model = sm.Logit(Y, X)
result = model.fit()

Optimization terminated successfully.
Current function value: 0.078584
Iterations 10
```

Resumen del modelo

```
result.summary()
```

Logit Regression Results

Dep. Variable:	default	No. Observations:	10000
Model:	Logit	Df Residuals:	9997
Method:	MLE	Df Model:	2
Date:	Fri, 31 May 2019	Pseudo R-squ.:	0.4619
Time:	18:16:05	Log-Likelihood:	-785.84
converged:	True	LL-Null:	-1460.3
		LLR p-value:	1.189e-293

	coef	std err	z	P> z	[0.025	0.975]
const	-10.7495	0.369	-29.115	0.000	-11.473	-10.026
student	-0.7149	0.148	-4.846	0.000	-1.004	-0.426
balance	0.0057	0.000	24.748	0.000	0.005	0.006

Figura 5.18

Realizando predicciones

```
result.predict(X.iloc[[9998]])

9998    0.148507
dtype: float64
```

Figura 5.19



6

Análisis de clúster

El análisis de clúster o análisis de conglomerados es un método estadístico no paramétrico, que tiene como objetivo hallar grupos a partir de los datos y que estos grupos estén claramente divididos. Se dice que el análisis de clúster es no paramétrico, porque no requiere que la distribución de la población sea caracterizada por ciertos parámetros, como en cambio sí ocurre con métodos como las regresiones lineal y logística.

Para agrupar las observaciones en un análisis de clúster, se deben comparar usando alguna medida de distancia o similitud entre ellas, según como sean los datos. Así existirán diferencias de enfoque en la metodología dependiendo de si las variables son cuantitativas o categóricas.

6.1. Supuestos del análisis de clúster.

Como el análisis de clúster es un método no paramétrico, las hipótesis estadísticas requeridas son menos exigentes que en el caso de una regresión lineal. Ahora bien, es necesario garantizar que cada variable a usar sea significativa y tener siempre presente que éstas pueden dividir de algún modo al conjunto de datos.

El análisis de clúster es sensible a los valores atípicos, por lo cual es indispensable corroborar o no la existencia de este tipo de datos, y en caso de detectar su presencia, manejarlos e interpretar adecuadamente su efecto en los resultados. Por lo tanto es recomendable

iniciar siempre el análisis de clúster haciendo un estudio de los valores atípicos.

6.2. Medidas o similitudes

Realizada la apropiada selección de las variables, los individuos bajo análisis vendrán representados por el conjunto de valores que tomen dichas variables en cada uno de ellos. Este es el punto de partida del análisis de clúster. Para clasificar adecuadamente los individuos es necesario determinar el grado de similaridad o disimilaridad (divergencia) entre ellos, de acuerdo a sus representaciones en el espacio de las variables. Existe una enorme cantidad de índices de similaridad y de disimilaridad o divergencia. Todos tienen propiedades y utilidades distintas que debemos conocer para aplicarlas correctamente a cada caso. La mayor parte de estos índices serán indicadores basados:

- En la distancia: considerando a los individuos como vectores en el espacio de las variables. En este sentido un elevado valor de la distancia entre dos individuos indicará un alto grado de disimilaridad entre ellos.
- En coeficientes de correlación.
- En tablas de datos de posesión o no de una serie de atributos.

6.3. Métodos jerárquicos y no jerárquicos

Dependiendo de cómo se realicen las agrupaciones, tendremos dos tipos de análisis de clúster: jerárquicos y no jerárquicos:

6.3.1. Métodos jerárquicos

Los llamados métodos jerárquicos tienen por objetivo agrupar clústeres para formar uno nuevo o separar alguno ya existente para dar origen a otros dos. Si sucesivamente se va efectuando el proceso de aglomeración o división, se minimizará alguna distancia o bien se maximizará alguna medida de similitud. Los métodos jerárquicos se subdividen en: aglomerativos y disociativos. Cada una de estas categorías presenta una gran diversidad de variantes

- Métodos aglomerativos: también conocidos como ascendentes, comienzan el análisis con tantos grupos como individuos haya. A partir de estas unidades iniciales se van formando grupos, de forma ascendente, hasta que al final del proceso todos los casos tratados están englobados en un mismo conglomerado.
- Métodos disociativos: también llamados descendentes, constituyen el proceso inverso al anterior. Comienzan con un conglomerado que engloba a todos los casos tratados y, a partir de este grupo inicial, a través de sucesivas divisiones, se van formando grupos cada vez más pequeños. Al final del proceso se tienen tantas agrupaciones como casos han sido tratados.

6.3.2. Métodos no jerárquicos

Los métodos jerárquicos para un conjunto de m individuos parten de m clústeres de un miembro cada uno hasta construir un solo clúster de m miembros (métodos aglomerativos), o viceversa (métodos disociativos). Los métodos que se presentan a continuación están diseñados para clasificar individuos (no son válidos para variables) en una clasificación de K clústeres, donde K se especifica a priori o bien se determina como una parte del proceso.

La idea central de la mayoría de estos procedimientos es elegir alguna partición inicial de individuos y después intercambiar los miembros de estos clústeres para obtener una mejor partición. Los diversos algoritmos existentes se diferencian sobre todo en lo que

se entiende por una mejor partición y en los métodos que deben usarse para conseguir mejoras.

6.4. Elección del número de grupos

Una pregunta natural es: ¿Cómo elegir de manera adecuada el número de grupos óptimos para segmentar la data? Usaremos básicamente dos formas visuales para escoger dicho número: el *dendrograma* y el *gráfico de codo*.

6.4.1. Dendrograma

Un dendrograma es un tipo de representación gráfica en forma de árbol que organiza y agrupa los datos en subcategorías según su similitud, dada por alguna medida de distancia. Los objetos similares se representan en el dendrograma por medio de un enlace cuya posición está determinada por el nivel de similitud entre los objetos o grupos de objetos. Dadas estas características, hace que los dendrogramas sean un tipo de diagrama muy útil para estudiar las agrupaciones de objetos; es decir, para estudiar los clústeres que pueden darse en un conjunto de datos. A continuación vemos el siguiente ejemplo de dendrograma:

A partir del dendrograma se puede apreciar un número apropiado de clúster.

6.4.2. Gráfico de codo

Este método considera el porcentaje de varianza explicado en función del número de conglomerados: Se debe elegir un número de clústeres que optimice el modelado, de modo que la mejora sea insignificante si se intenta añadir otro clúster. En otros términos, si se traza el porcentaje de varianza explicado por los clústeres contra

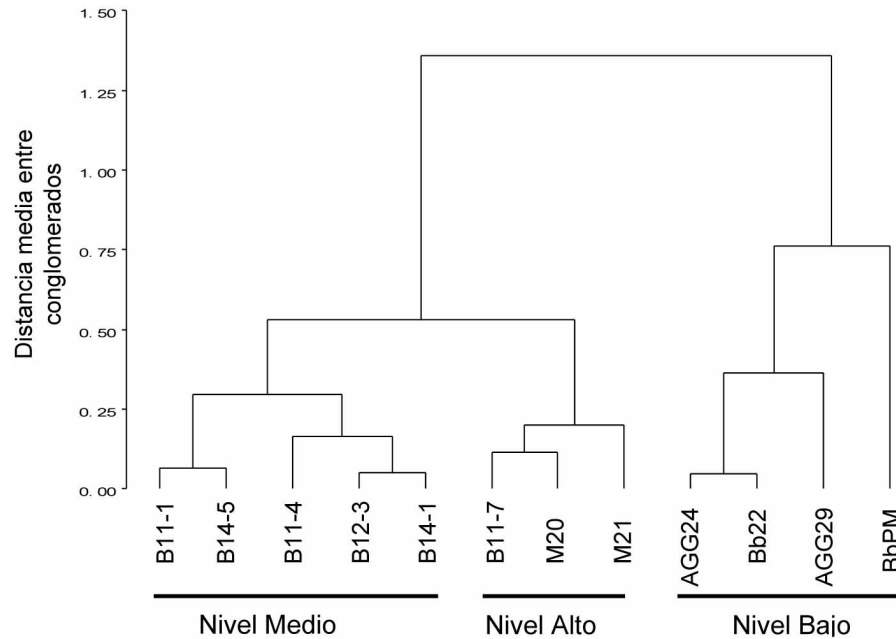


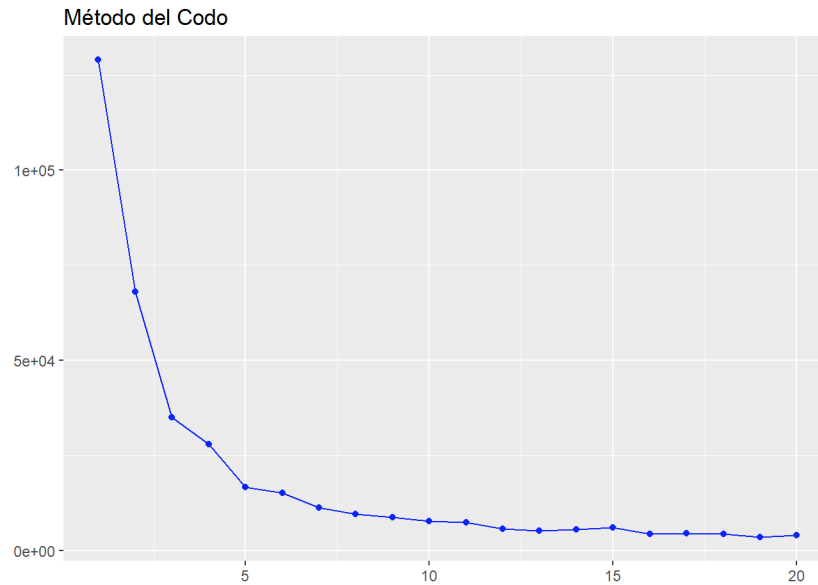
Figura 6.1

el número de clústeres, los primeros clústeres añadirán mucha información (explican mucha varianza), pero en algún momento la ganancia marginal producto de añadir clústeres adicionales caerá, mostrando un ángulo en el gráfico. El número de grupos se elige justo hasta este punto, de ahí el “criterio del codo”. Este “codo” no siempre puede ser identificado inequívocamente. La siguiente imagen nos proporciona una muestra de un gráfico de codo:

6.5. Análisis de clúster en R

6.5.1. Método no jerárquico

Veremos el método llamado *K-medias* que en resumidas palabras agrupa un número de clústeres pre-definidos a partir de los valores medios de cada clúster. Usaremos el conjunto de datos *iris* con

**Figura 6.2**

información sobre tres variedades morfológicas de la especie iris, el cual viene por defecto en *R*, también la función *kmeans()* la cual requiere dos parámetros: el conjunto de datos y el número de clústeres a crear.

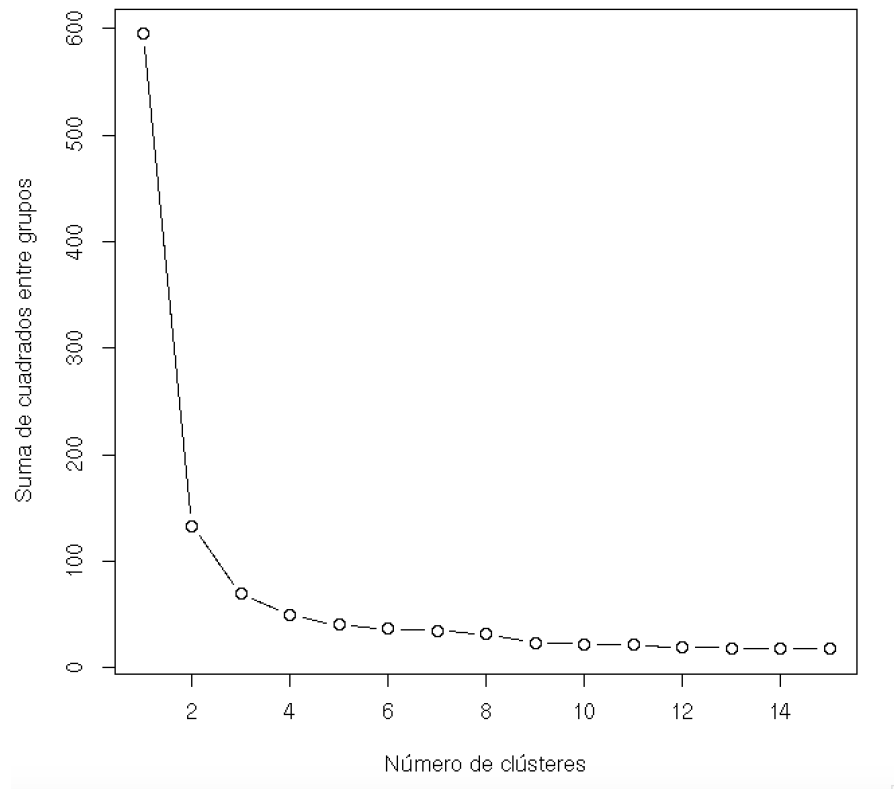
Lo primero que haremos es calcular un diagrama de codo para poder tener una primera impresión del número de grupos a utilizar, esto lo haremos con el siguiente código:

```
mydata = iris[1:3]

wss <- (nrow(mydata)-1)*sum(apply(mydata,2,var))
for (i in 2:15) wss[i] <- sum(kmeans(mydata,
                                   centers=i)$withinss)
plot(1:15, wss, type="b", xlab="Número de clústeres",
     ylab="Suma de cuadrados entre grupos")
```

Figura 6.3

Obteniendo la siguiente imagen:

**Figura 6.4**

A partir de este gráfico podemos intuir que el número de clústeres adecuados es 3, luego para construir los grupos usaremos el siguiente código:

```
#Ejemplo de Cluster

data(iris)

#Generamos 5 grupos en nuestros datos
modelo5<-kmeans(iris[1:3] , centers = 3)

plot(iris[1:2],col=modelo5$cluster,main="Ejemplo de 3 clústeres",xlab="longitud del sépalo",ylab="Ancho del sépalo")
```

Figura 6.5

Para tener una visión gráfica de los clústeres hacemos proyecciones en pares de variables, obteniendo el siguiente gráfico:

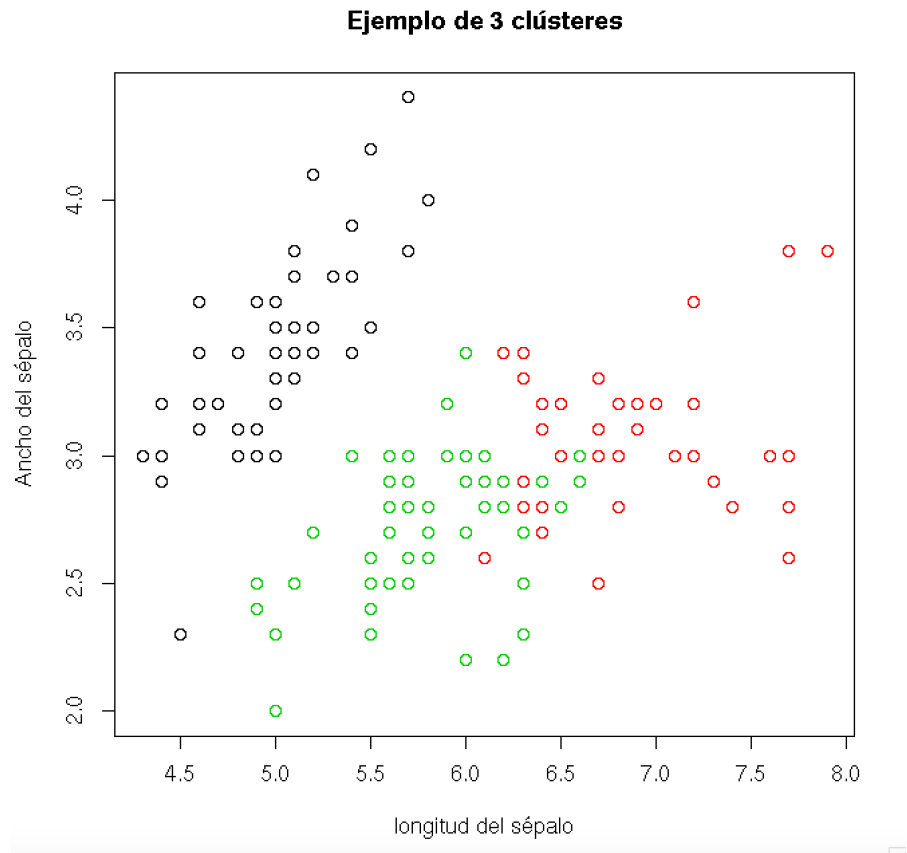


Figura 6.6

6.5.2. Método jerárquico

Para construir los grupos, primero debemos crear las distancias entre observaciones, esto lo haremos con la función *dist()*. Esta función tiene como parámetros el conjunto de datos y la manera en cómo se medirán los datos, en nuestro caso con la distancia euclídea. Luego con la función *hclust()* creamos el proceso de agrupación entre individuos, ésta función necesita dos parámetros: las distan-

cias calculadas y el método de agrupación entre los individuos. En nuestro caso usaremos el método “*complete*”. A continuación realizaremos el gráfico de dendrograma. Veamos esto en el siguiente código:

```
# Creando las distancias entre observaciones
d <- dist(iris[1:3], method = "euclidean")

# Análisis jerárquicos
hc1 <- hclust(d, method = "complete" )

# Gráfica del dendrograma
plot(hc1, cex = 0.6, hang = -1, main="Dendrograma", xlab = "Observaciones", ylab="Número de división")
```

Figura 6.7

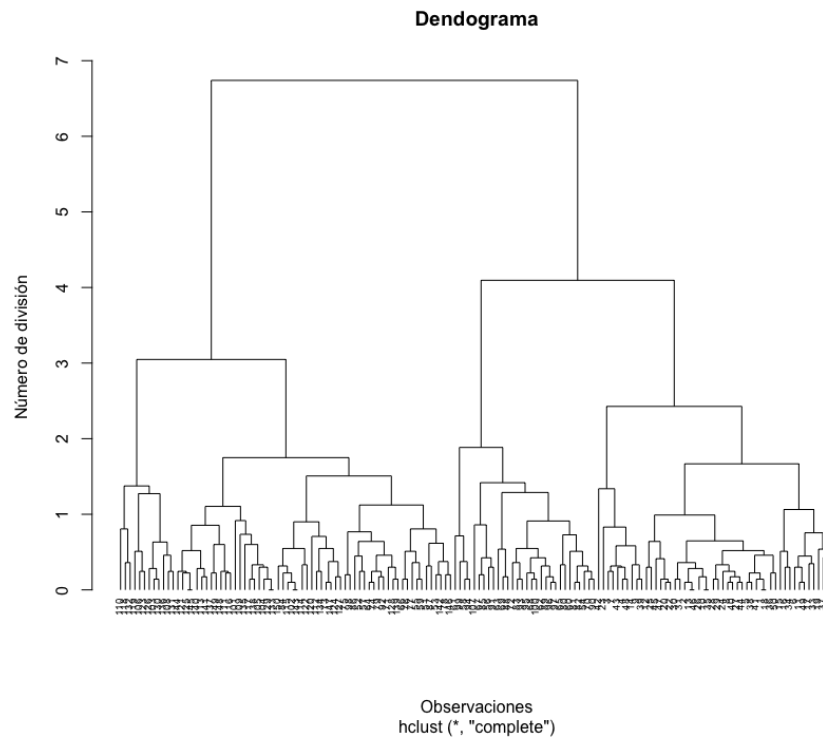


Figura 6.8

Con este dendrograma podemos apreciar una posible separación

de la variable de entre 3 o 4 grupos. Para obtener estos grupos, usaremos la función `cutree()` la cual recibe como parámetros el modelo y el número de grupos a escoger, a continuación se muestra este procedimiento:

```
# Escogiendo el numero de cluster
grupos <- cutree(hc1, k = 4)

# Observando la distribución por grupos

table(grupos)
```

Figura 6.9

6.6. Análisis de clúster en *Python*

6.6.1. Método no jerárquico

Realizaremos la construcción de los grupos usando el método *kmeans* con el mismo conjunto de datos. No describiremos en detalle las librerías a utilizar, solo describiremos su uso general, utilizando la librería *sklearn* para crear el modelo y *seaborn*, *matplotlib* y *mpl_toolkits* para la parte gráfica.

Ahora construiremos el gráfico de codos para poder determinar el número apropiado de clústeres. Usaremos la función *Kmeans* de la librería *sklearn* para construir los modelos por número de clústeres. Seguidamente se muestra el siguiente código para este propósito.

Obteniendo el siguiente resultado:

A partir de este gráfico creamos un número de tres grupos, con la función *kmean()*. Con *.predicts()* identificamos el grupo al cual pertenece cada observación.

Librerías a usar

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin_min
%matplotlib inline
from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['figure.figsize'] = (16, 9)
plt.style.use('ggplot')
```

Conjunto de datos

```
datos = pd.read_csv("datos_cluster.csv", index_col=0)
```

Figura 6.10

Construcción del gráfico de codo

```
Nc = range(1, 20)
kmeans = [KMeans(n_clusters=i) for i in Nc]
score = [kmeans[i].fit(datos).score(datos) for i in range(len(kmeans))]
plt.plot(Nc, score)
plt.xlabel('Número de clúster')
plt.ylabel('Score')
plt.title('Gráfico de codo')
plt.show()
```

Figura 6.11

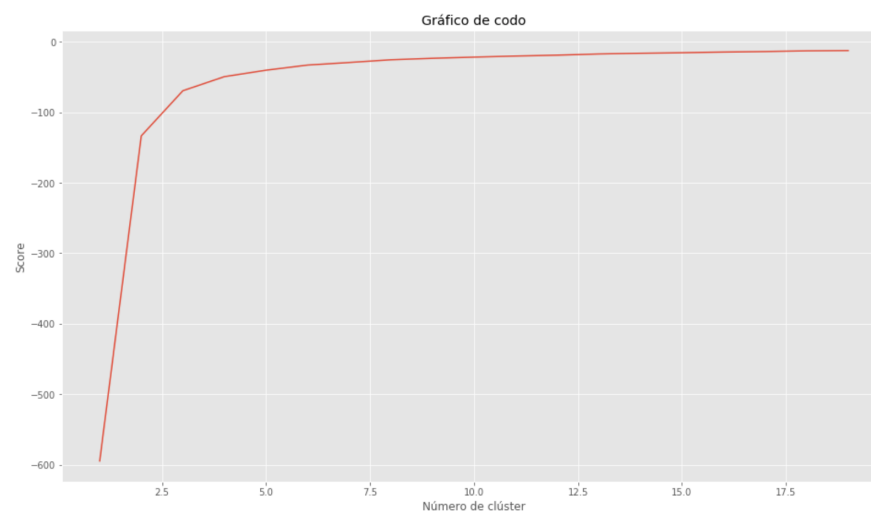


Figura 6.12

Construyendo los grupos

```
kmeans = KMeans(n_clusters=3).fit(datos)
```

Grupos

```
labels = kmeans.predict(datos)
```

Figura 6.13

6.6.2. Método jerárquico

Para el cálculo de los grupos usaremos el mismo conjunto de datos, una vez cargados construimos el dendrograma, para lo cual usaremos la función *.dendrogram()* de la librería *scipy*. En este ejemplo usaremos el método de *ward*. A continuación mostramos el código:

Clústeres jerárquicos

```
cluster = pd.read_csv("datos_cluster.csv", index_col=0)
```

Construyendo el dendrograma

```
import scipy.cluster.hierarchy as sch

dendrogram = sch.dendrogram(sch.linkage(cluster, method = 'ward'))

plt.title('Dendrograma')
plt.xlabel('Clientes')
plt.ylabel('Distancias Euclidianas')
plt.savefig("Dendrograma.png")
plt.show()
```

Figura 6.14

Este código nos da como resultado :

Al observar el dendrograma observamos la existencia de tres grupos posibles. Para obtenerlos usaremos la función *AgglomerativeClustering()* de la librería *sklearn* y finalmente *.fit_predict()* para obtener los grupos. El procedimiento a seguir se presenta a continuación:

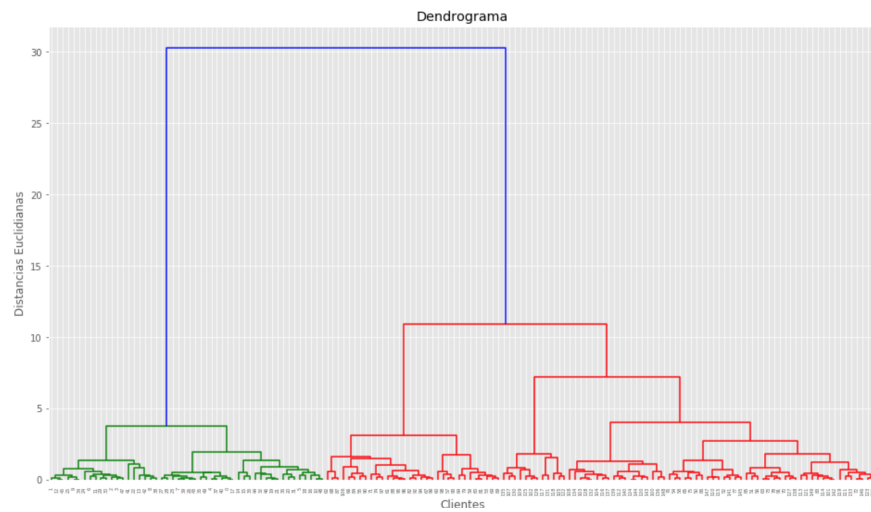


Figura 6.15

Obtener los clústeres de los datos

```
from sklearn.cluster import AgglomerativeClustering

hc = AgglomerativeClustering(n_clusters = 3,
                             affinity = 'euclidean',
                             linkage = 'ward')

y_hc = hc.fit_predict(cluster)
y_hc
```

Figura 6.16



7

Análisis de Componentes Principales

El análisis de componentes principales o PCA por sus siglas en inglés (Principal component analysis), es un método concebido para la reducción de variables que conforman un conjunto de datos. La idea fundamental detrás de la matemática en la que se fundamenta, es crear a través de las variables, un nuevo grupo de variables que tengan muy poca correlación entre sí, de modo que con una cantidad reducida de ellas se pueda explicar la mayor cantidad de varianza del conjunto de datos. Cada variable nueva es llamada componente, en el cual el orden representa la cantidad de varianza explicada, de modo que el nivel de varianza explicada decrece entre mayor sea el orden de la componente a utilizar. Ésta es la clave a la hora de reducir variables, pues en general se escoge un nivel de componentes que explique una cantidad considerable de varianza, por ejemplo, el 80 %.

7.1. Correlación o Covarianza:

A la hora de calcular las componentes principales debemos especificar la matriz a utilizar para dichos cálculos, es decir, la matriz de correlación o la matriz de covarianza. El criterio general para dicha escogencia es el siguiente:

- Matriz de correlación: Cuando los datos no son dimensionalmente homogéneos o el orden de magnitud de las variables medidas no es el mismo.
- Matriz de covarianza: Cuando los datos son dimensionalmente

homogéneos y presentan valores medios similares. En general, se suelen normalizar los datos, es decir, dividir por su varianza y trasladar su valor medio al origen, en este punto se suele usar en la mayoría de los casos la matriz de covarianza.

7.2. Supuestos estadísticos

Este análisis exige verificar ciertos supuestos estadísticos:

- Se asume que los datos observados siguen cierta linealidad a través de cierta base de $\mathbb{R}^n$. Por ejemplo, debemos asegurarnos de que las variables estén lo menos correlacionadas entre sí.
- El *PCA* utiliza los vectores propios de la matriz de covarianzas o correlación y sólo encuentra las direcciones de ejes en el espacio de variables considerando que los datos se distribuyen de manera normal.

7.3. *PCA* en *R*

7.3.1. Librerías y datos

Usaremos la base de datos *USArrests* la cual viene por defecto en *R* y contiene información acerca de ciertos índices delictivos en EEUU.

Al cargar los datos calcularemos las medias y varianzas de las variables para saber si es necesario normalizar los datos. Para ello empleamos el siguiente código:

El primer resultado es la media por variable, observándose considerables diferencias

```
#### Datos
datos = data("USArrests")

### cálculo de medias por variables
apply(X = USArrests, MARGIN = 2, FUN = mean)

### cálculo de varianzas por variables

apply(X = USArrests, MARGIN = 2, FUN = var)
```

Figura 7.1

Murder	Assault	UrbanPop	Rape
7.788	170.760	65.540	21.232

Figura 7.2

De igual manera a continuación podemos observar considerables diferencias en las varianzas entre las variables:

Murder	Assault	UrbanPop	Rape
18.97047	6945.16571	209.51878	87.72916

Figura 7.3

7.3.2. Cálculo de componentes

Ahora, con la función *prcomp()* calculamos las componentes principales. Esta función recibe como parámetros los datos y la variable *scale*, la cual de ser *TRUE* normaliza los datos y de ser *FALSE* los deja en su forma original. Una vez hecho lo anterior podemos a través del atributo *rotation* ver los datos en función de las nuevas componentes.

```

### Cálculo de las componentes principales

pca <- prcomp(USArrests, scale = TRUE)

### Mostrando la nueva matriz de datos
pca$rotation

```

Figura 7.4

	PC1	PC2	PC3	PC4
Murder	-0.5358995	0.4181809	-0.3412327	0.64922780
Assault	-0.5831836	0.1879856	-0.2681484	-0.74340748
UrbanPop	-0.2781909	-0.8728062	-0.3780158	0.13387773
Rape	-0.5434321	-0.1673186	0.8177779	0.08902432

Figura 7.5

7.3.3. Funciones gráficas

Ahora para escoger el número de componentes utilizaremos el gráfico de codo. Para ello debemos calcular previamente la varianza por componente. Además, usaremos el paquete *ggplot* para obtener una gráfica más elegante. A continuación se muestra este proceso:

Con el gráfico anterior podemos intuir que con tres componentes obtendremos una gran parte de la varianza explicada.

7.4. PCA en Python

7.4.1. Librerías y datos

Procederemos a construir el modelo en *Python*, primero cargaremos las librerías y datos a usar, usaremos *sklearn* para construir el modelo y los mismos datos de la sección anterior

```
### Librería para graficar el codo de varianzas
library(ggplot2)

## Código para la varianza
prop_varianza <- pca$sdev^2 / sum(pca$sdev^2)
prop_varianza_acum <- cumsum(prop_varianza)

ggplot(data = data.frame(prop_varianza_acum, pc = 1:4),
      aes(x = pc, y = prop_varianza_acum, group = 1)) +
  geom_point() +
  geom_line() +
  theme_bw() +
  labs(x = "Componente principal",
       y = "Prop. varianza explicada acumulada")
```

Figura 7.6

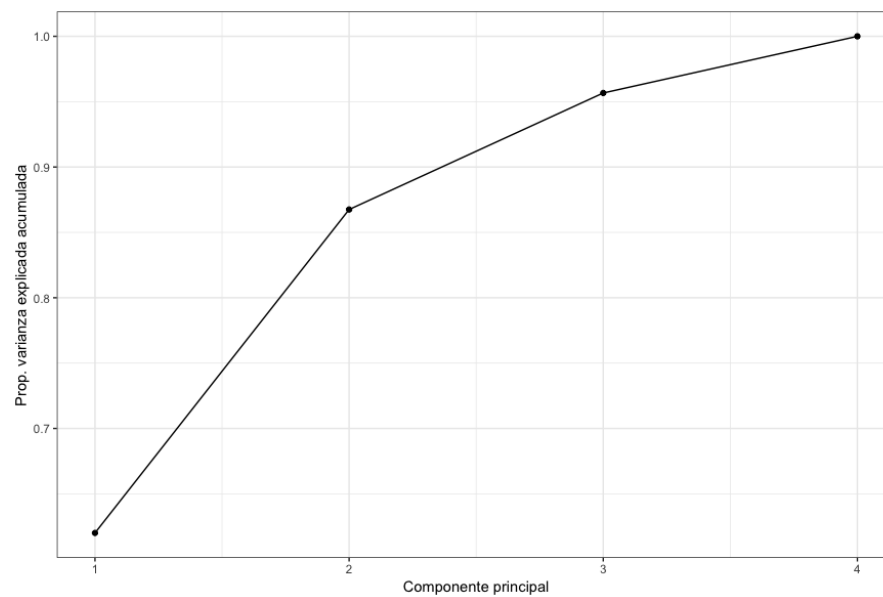


Figura 7.7

PCA

Paquetes y datos

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin_min
%matplotlib inline
from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['figure.figsize'] = (16, 9)
plt.style.use('ggplot')

datos = pd.read_csv("pca.csv", index_col=0)
```

Figura 7.8

7.4.2. Cálculo de componentes

Procedemos a ver la estructura de los datos, usando las funciones `.mean()` y `.var()` de *Pandas* para calcular la media y varianza de cada variable del conjunto de datos.

Media y varianza

```
datos.mean()

Murder      7.788
Assault    170.760
UrbanPop    65.540
Rape       21.232
dtype: float64

datos.var()

Murder      18.970465
Assault   6945.165714
UrbanPop   209.518776
Rape       87.729159
dtype: float64
```

Figura 7.9

Observamos que los datos tienen escalas muy diferentes, por lo que procedemos a realizar la normalización de los datos, usando la función `scale()` de *Sklearn*. Una vez tenemos los datos con la misma escala, usamos las funciones `PCA()` y `.fit()` para construir el modelo. Finalizamos usando la función `.transform()` para convertir los datos originales a las nuevas componentes calculadas.

Normalizando los datos

```
d = iris[["Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width"]]
from sklearn.preprocessing import scale
data=pd.DataFrame(scale(d))
```

Calculando componentes

```
from sklearn.decomposition import PCA
pca=PCA().fit(d)
pca_samples=pca.transform(d)
```

Figura 7.10**7.4.3. Funciones Gráficas**

Como este curso no es de programación, anexaremos directamente el siguiente código sin mayores detalles, con el proposito de realizar las gráficas correspondientes del modelo:

Función para graficar componentes

```
def pca_results(good_data, pca):
    """
    Create a DataFrame of the PCA results
    Includes dimension feature weights and explained variance
    Visualizes the PCA results
    """

    #Indexando las dimensiones
    dimensions = ['Dimension {}'.format(i) for i in range(1,len(pca.components_)+1)]

    # Componentes PCA
    components = pd.DataFrame(np.round(pca.components_, 4), columns = list(good_data.keys()))
    components.index = dimensions

    # Varianza explicada PCA
    ratios = pca.explained_variance_ratio_.reshape(len(pca.components_), 1)
    variance_ratios = pd.DataFrame(np.round(ratios, 4), columns = ['Explained Variance'])
    variance_ratios.index = dimensions

    # Crear una visualización en gráfico de barra
    fig, ax = plt.subplots(figsize = (14,8))

    # graficar la característica Pesos como una función de las componentes
    components.plot(ax = ax, kind = 'bar');
    ax.set_ylabel("Feature Weights")
    ax.set_xticklabels(dimensions, rotation=0)

    # Mostrar los ratios de la varianza explicada
    for i, ev in enumerate(pca.explained_variance_ratio_):
        ax.text(i-0.40, ax.get_ylim()[1] + 0.05, "Explained Variance\n%.4f"%(ev))

    # Obtener un DataFrame concatenado
    return pd.concat([variance_ratios, components], axis = 1)
```

Figura 7.11

A continuación anexamos el gráfico de codo:

Con el gráfico de codo podemos apreciar que con 2 o 3 componentes tenemos una gran cantidad de varianza explicada.

Gráfico de codo

```
plt.plot(list(range(1, len(pca.explained_variance_ratio_)+1)), np.cumsum(pca.explained_variance_ratio_))  
plt.xlabel('Numero de Componente')  
plt.ylabel('Varianza acumulada explicada')  
plt.show()
```

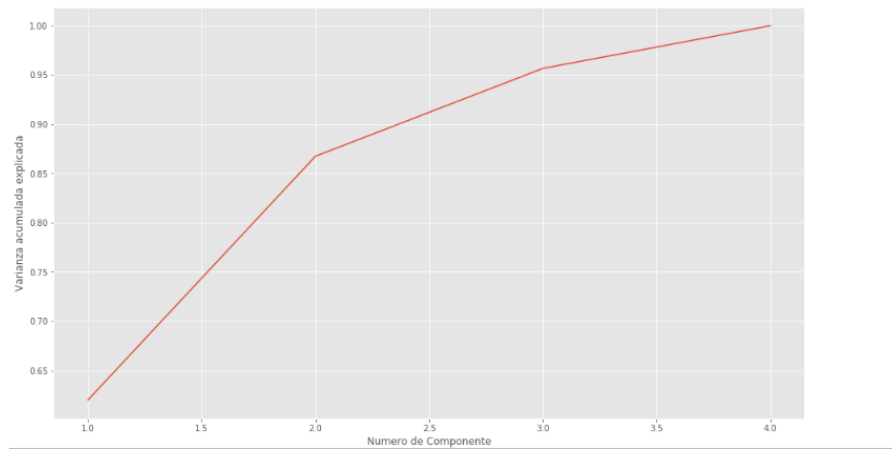


Figura 7.12

8

Árboles de decisión y bosques aleatorios

8.1. Árboles de decisión

Los árboles de decisión son una herramienta muy útil a la hora de elegir entre varias alternativas, cuando estas son escalonadas, es decir, que los resultados (positivos o negativos) dependen de una decisión tomada anteriormente. Independientemente del área en que sean utilizados, los árboles de decisión facilitan la visualización y el análisis de las probabilidades de los resultados entre diferentes opciones, mejorando la capacidad de las empresas e individuos para realizar una mejor elección.

Un árbol de decisión es, para quien va a tomar la decisión, un modelo esquemático de las alternativas disponibles y de las posibles consecuencias de cada una. El nombre de la herramienta proviene de la forma que adopta el modelo, el cual semeja un árbol. El modelo está conformado por múltiples nodos cuadrados que representan puntos de decisión. De estos puntos surgen ramas (que deben leerse de izquierda a derecha), las cuales representan las distintas alternativas. Las ramas que provienen de los nodos circulares o causales representan los eventos. A continuación presentamos un ejemplo gráfico de un árbol de decisión:

8.1.1. Árboles de decisión en *R*

Para realizar la construcción de árboles de decisión usaremos la librería *C50*, junto con la función *C5.0()* con una sintaxis similar a la de la regresión lineal, es decir, ingresamos la variable a predecir

Partes del árbol

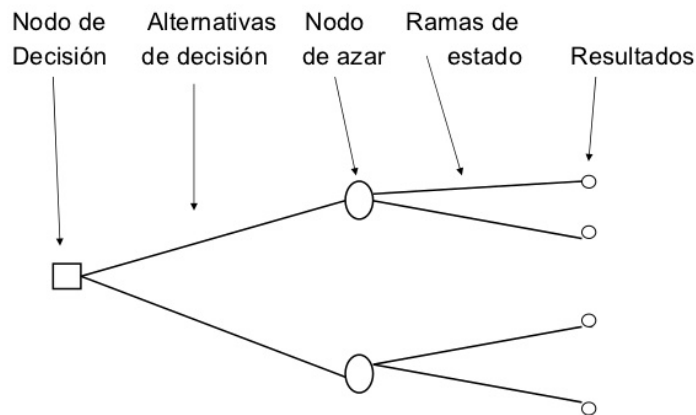


Figura 8.1

seguida de las variables independientes. El segundo parámetro son los datos que se usarán. A continuación se muestra este proceso:

```
library(C50)

Data = iris

Arbol = C5.0(Species ~., Data)

plot(Arbol)
```

Figura 8.2

Al usar la función `plot()` obtenemos la gráfica del árbol de decisión calculado.

Para hacer predicciones, usaremos la función `predict()` la cual

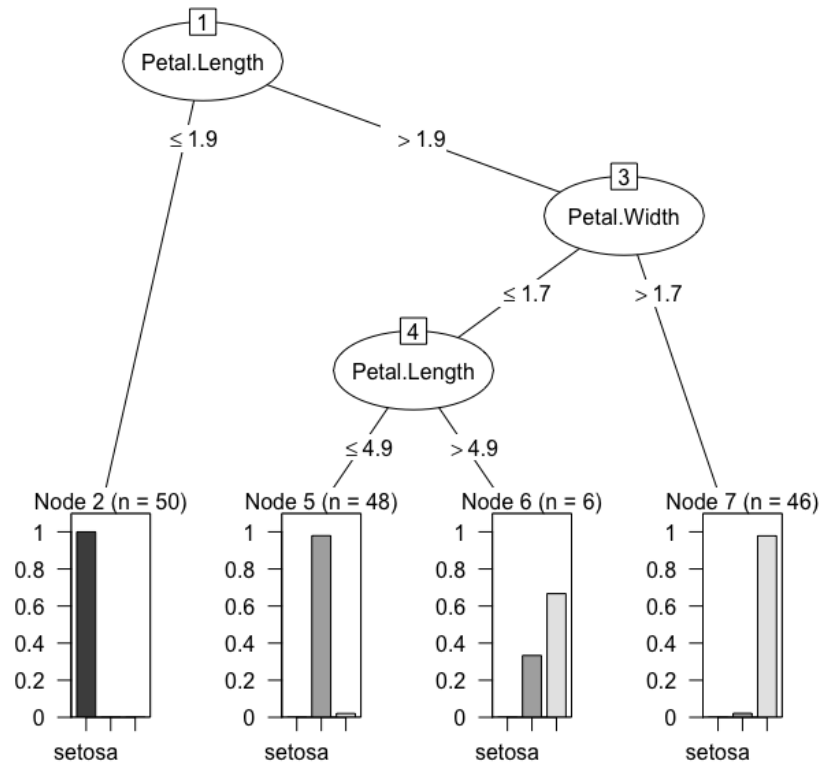


Figura 8.3

necesita como argumentos el nombre del modelo y los datos a predecir.

Para ello construimos unos posibles datos para predecir los resultados del modelo “*Arbol*”, como se muestra en el siguiente ejemplo:

8.1.2. Árboles de decisión en *Python*

Usaremos la librería *sklearn* y los datos *iris*. Se debe separar la variable a predecir de la variable independiente antes de realizar la construcción del modelo.

Construimos el modelo usando usando la función *DecisionTreeClassifier()* y luego lo entrenamos usando usando la función *.fit()*

```
> data1 = c(6.7, 3.1, 5.4, 2.1)
>
> data1 =(as.data.frame(data1))
>
> data1 = t(data1)
>
> colnames(data1) = colnames(iris[,-c(5)])
>
> data1 <- as.data.frame(data1)
>
>
> prediccion <- predict(Arbol, data1)
>
> prediccion
[1] virginica
Levels: setosa versicolor virginica
```

Figura 8.4

Librerías y datos

```
from sklearn.tree import DecisionTreeClassifier

iris = pd.read_csv("arbol.csv", index_col=0)

predictores = iris[["Sepal.Length", "Sepal.Width", "Petal.Width"]]
objetivo = iris[["Species"]]
```

Figura 8.5

agregando como parámetros la variable independiente y las variables independientes.

Construcción del modelo

```
Arbol = DecisionTreeClassifier()
Arbol = Arbol.fit(predictores, objetivo)
```

Figura 8.6

Para realizar predicciones usamos la función `.predict()`

Predicciones

```
predicciones = Arbol.predict(predictores)
```

Figura 8.7

8.2. Bosques aleatorios

Bosques aleatorios es una herramienta que combina árboles de decisión, seleccionando de manera aleatoria una cantidad de variables con las cuales se construye cada uno de los árboles individuales, y se realizan predicciones con estas variables que posteriormente serán ponderadas a través del cálculo de la clase más votada de los árboles que se generaron, para finalmente hacer la predicción. El método promedia una colección de árboles no correlacionados, o decorrelacionados, y dónde cada árbol depende de los valores de un vector aleatorio de la muestra de manera independiente y con la misma distribución de todos los árboles en el bosque. La generalización del error para los bosques converge a un límite en cuanto el número de árboles en el bosque sea grande. El error de generalización de un bosque de árboles de clasificación depende de la fuerza de los árboles individuales en el bosque y la correlación entre ellos.

8.3. Bosques aleatorios en R

Para realizar bosques aleatorios en R usaremos la librería *randomForest* la cual facilita la sintaxis necesaria para construir el modelo. Los datos corresponden a la base de datos *iris* y se debe separar la variable independiente de las dependientes para luego usar la función *randomForest()* para construir el modelo. Ésta función necesita como parámetros las variables independientes y la variable

dependiente junto al número de árboles a promediar. Anexamos el código para este proceso:

```
library(randomForest)

Predictoras = iris[, 1:4]
Objetivo = iris[, 5]

rf <- randomForest(x = Predictoras, y = Objetivo, ntree = 500)

print(rf)
```

Figura 8.8

Al realizar el *print* del modelo obtenemos:

```
Call:
  randomForest(x = Predictoras, y = Objetivo, ntree = 500)
      Type of random forest: classification
      Number of trees: 500
No. of variables tried at each split: 2

      OOB estimate of  error rate: 4.67%
Confusion matrix:
      setosa versicolor virginica class.error
setosa      50         0         0       0.00
versicolor   0        47         3       0.06
virginica     0         4        46       0.08
```

Figura 8.9

Aquí obtenemos la información sobre cómo se construyó el modelo y la clasificación de los datos. En el ejemplo notamos que el modelo es muy preciso. Las predicciones la hacemos de la manera habitual, usando la función *predict()* incorporando los parámetros correspondientes al nombre del modelo y al conjunto de nuevos datos:

```
predict(rf, iris[, 1:4])
```

Figura 8.10

8.4. Bosques aleatorios en *Python*

Usaremos de igual manera la librería *sklearn*, a modo de conocimiento cargaremos los datos *iris* desde *sklearn*, pero tendremos que hacerles unas modificaciones previas, de la siguiente forma:

Librerías y datos

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
```

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Debemos separar las variables independientes, en otra variable que llamaremos *predictores*. Como los datos carecen de la variable *species* la agregaremos de la siguiente forma:

Antes de construir el modelo, aseguramos que la variable sea del tipo numérico con la función *factorize*:

Para crear el modelo usamos la función *Random-*

```
predictores = df.columns[:4]
```

```
# Add a new column with the species names, this is what we are going to try to predict
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

# View the top 5 rows
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Figura 8.11

```
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
objetivo = pd.factorize(df['species'])[0]

objetivo
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2,  
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,  
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Figura 8.12

ForestClassifier() y *.fit()* para entrenar el modelo.

Creando el modelo

```
clf = RandomForestClassifier(n_jobs=2, random_state=0)
clf.fit(df[predictores], objetivo)
```

Finalmente, con *.predict()* realizamos predicciones con nuestro modelo.

Predicciones

```
clf.predict(df[predictores])

array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Figura 8.13



A

Herramientas de software

Para aquellos que no están familiarizados con los paquetes de software necesarios para usar R Markdown, damos una breve introducción sobre la instalación y mantenimiento de estos paquetes.

A.1. *R* y paquetes de *R*

R puede descargarse e instalarse desde cualquier réplica de CRAN (Comprehensive R Archive Network), por ejemplo de <https://cran.rstudio.com>. Tenga en cuenta que existirán algunas nuevas versiones de *R* cada año, y es posible que desee actualizar *R* de vez en cuando.

Para instalar el paquete *bookdown*, puede escribir en R:

```
install.packages("bookdown")
```

De esta manera se instalan todos los paquetes *R* requeridos. También puede elegir instalar todos los paquetes opcionales, si no le importa demasiado si estos paquetes serán usados realmente para compilar su libro (como **htmlwidgets**):

```
install.packages("bookdown", dependencies = TRUE)
```

Si quiere probar la versión de desarrollo de **bookdown** en *GitHub*, primero tiene que instalar **devtools**:

```
if (!requireNamespace('devtools')) install.packages('devtools')
devtools::install_github('rstudio/bookdown')
```

Los paquetes *R* también se actualizan constantemente en *CRAN* o *GitHub*, por lo que es posible que desee actualizarlos de vez en cuando:

```
update.packages(ask = FALSE)
```

Aunque no es necesario, el IDE de RStudio puede hacer muchas cosas mucho más fáciles cuando se trabaja en proyectos relacionados con R. El IDE de RStudio se puede descargar desde <https://www.rstudio.com>

A.2. Pandoc

Un documento R Markdown (*.Rmd*) es primero compilado a Markdown (*.md*) a través del paquete *knitr*, y luego *Markdown* es compilado a otros formatos de salida (como *LaTeX* o *HTML*) a través de *Pandoc*. Este proceso está automatizado a través del paquete *rmarkdown*. No es necesario instalar *knitr* o *rmarkdown* por separado, ya que son los paquetes requeridos de *bookdown* y se instalarán automáticamente cuando instale *bookdown*. Sin embargo, *Pandoc* no es un paquete *R*, por lo que no se instalará automáticamente cuando instale *bookdown*. Puede seguir las instrucciones de instalación en la página principal de Pandoc (<http://pandoc.org>) para instalar *Pandoc*, pero si utiliza el IDE de *RStudio*, no necesita instalar *Pandoc* por separado, porque *RStudio* incluye una copia de Pandoc. El número de versión de *Pandoc* se puede obtener a través de:

```
rmarkdown::pandoc_version()
## [1] '2.3.1'
```

Si encuentra esta versión demasiado baja y hay características de *Pandoc* sólo en una versión posterior, puede instalar la versión posterior de *Pandoc*, y *rmarkdown* llamará a la versión más reciente en lugar de su versión incorporada.

A.3. *LaTeX*

LaTeX es necesario sólo si desea convertir su libro a *PDF*. La elección típica de la distribución *LaTeX* depende de su sistema operativo. Los usuarios de *Windows* pueden considerar *MiKTeX* (<http://miktex.org>), los usuarios de *Mac OS X* pueden instalar *MacTeX* (<http://www.tug.org/mactex/>), y los usuarios de *Linux* pueden instalar *TeXLive* (<http://www.tug.org/texlive>). Visite <https://www.latex-project.org/get/> para obtener más información sobre *LaTeX* y su instalación.

La mayoría de las distribuciones *LaTeX* ofrecen un paquete mínimo/básico y un paquete completo. Puede instalar el paquete básico si tiene poco espacio de disco y sabe cómo instalar los paquetes *LaTeX* más tarde. El paquete completo es a menudo significativamente más grande en tamaño, ya que contiene todos los paquetes de *LaTeX*, y es poco probable que se encuentre con el problema de que le falten paquetes en *LaTeX*.

Los mensajes de error de *LaTeX* pueden ser oscuros para los principiantes, pero puede encontrar soluciones buscando el mensaje de error en línea (tiene buenas posibilidades de terminar en *StackExchange*). De hecho, el código *LaTeX* convertido a partir de *R Markdown* debería ser lo suficientemente seguro y no debería tener problemas frecuentes con *LaTeX* a menos que haya introducido contenido *LaTeX* en bruto en sus documentos *Rmd*. El problema

más común de *LaTeX* debería ser la falta de paquetes *LaTeX*, y el error puede parecerse a esto:

```
! LaTeX Error: File `titling.sty' not found.

Type X to quit or <RETURN> to proceed,
or enter new name. (Default extension: sty)

Enter file name:
! Emergency stop.
<read *>

1.107 ^~M

pandoc: Error producing PDF
Error: pandoc document conversion failed with error 43
Execution halted
```

Esto significa que usó un paquete que contiene *titling.sty*, pero no fue instalado. Los nombres de los paquetes *LaTeX* suelen ser los mismos que los de los archivos *.sty*, por lo que en este caso, puede intentar instalar el paquete de titulación. Tanto *MiKTeX* como *MacTeX* proporcionan una interfaz gráfica de usuario para gestionar paquetes. Puede encontrar el administrador de paquetes de *MiKTeX* en el menú de inicio, y el administrador de paquetes de *MacTeX* en la aplicación “*TeX Live Utility*”. Escriba el nombre del paquete o el nombre del archivo para buscar el paquete e instalarlo. *TeXLive* puede ser un poco más complicado: si usa los paquetes *TeXLive* preconstruidos de su distribución de *Linux*, necesita buscar en el repositorio de paquetes y sus palabras clave pueden coincidir con otros paquetes que no sean de *LaTeX*. A veces resulta frustrante usar las colecciones de paquetes predefinidas en *Linux*, y es mucho más fácil instalar *TeXLive* desde el código fuente, en cuyo caso puede administrar paquetes usando el comando *tlmgr*. Por ejemplo, puede buscar *titling.sty* desde el repositorio de paquetes de *TeXLive*:

```
tlmgr search --global --file titling.sty
# titling:
# texmf-dist/tex/latex/titling/titling.sty
```

Una vez que haya averiguado el nombre del paquete, puede instalarlo por medio de:

```
tlmgr install titling # may require sudo
```

Las distribuciones y paquetes de *LaTeX* también se actualizan de vez en cuando, y puede considerar actualizarlos especialmente cuando tenga problemas con *LaTeX*. Puede encontrar la versión de su distribución *LaTeX* por:

```
system('pdflatex --version')
## pdfTeX 3.14159265-2.6-1.40.19 (TeX Live 2018)
## kpathsea version 6.3.0
## Copyright 2018 Han The Thanh (pdfTeX) et al.
## There is NO warranty. Redistribution of this software is
## covered by the terms of both the pdfTeX copyright and
## the Lesser GNU General Public License.
## For more information about these matters, see the file
## named COPYING and the pdfTeX source.
## Primary author of pdfTeX: Han The Thanh (pdfTeX) et al.
## Compiled with libpng 1.6.34; using libpng 1.6.34
## Compiled with zlib 1.2.11; using zlib 1.2.11
## Compiled with xpdf version 4.00
```

